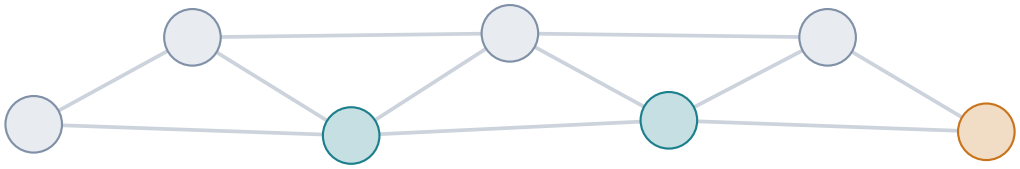
[COLLEGE / INSTITUTE NAME]

[Department Name]

LABORATORY MANUAL & RECORD

COMPUTER NETWORKS LABORATORY

Network Simulation with NS-2 • Data-Link Layer and Socket Programs in Python



Name	[Student Name]
USN / Roll No.	[USN / Roll No.]
Semester	[Semester]
Section / Batch	[Section / Batch]
Academic Year	2026–27
Faculty In-charge	[Faculty In-charge]

Experiments 1–10 • NS-2 • AWK • XGraph • Python • C

[COLLEGE / INSTITUTE NAME]
[Department Name]

C E R T I F I C A T E

This is to certify that **[Student Name]**, bearing USN / Roll No. **[USN / Roll No.]**, of [Semester] semester ([Section / Batch]), has satisfactorily completed all the experiments prescribed for the **Computer Networks Laboratory** in the academic year 2026–27, as recorded in this manual.

[Faculty In-charge]
Faculty In-charge

Head of the Department
[Department Name]

Date: _____

Place: _____

Index

Sl. No.	Date	Name of the experiment	Page	Marks	Signature
1	14/08/2026	Three-node point-to-point network: queue size, bandwidth and packet drops	3		
2	21/08/2026	Four-node network with TCP and UDP traffic: packet counts and throughput	7		
3	28/08/2026	Ethernet LAN: throughput versus data rate and error rate	13		
4	04/09/2026	Ethernet LAN with multiple traffic flows: congestion window of TCP Vegas and Reno	18		
5	11/09/2026	Bit stuffing in the data-link layer	23		
6	09/10/2026	Error detection using CRC (CRC-CCITT)	26		
7	07/10/2026	Frame sorting technique used in buffers	30		
8		Client–server file transfer using TCP sockets	33		
9		Client–server calculator using UDP sockets	38		
10	06/11/2026	Distance vector routing algorithm	44		

Appendix A (viva voce question bank) is on page 48; Appendix B (troubleshooting guide) is on page 51; Appendix C (files and quick command reference) is on page 53.

INTRODUCTION

NS-2 and the Experimental Workflow

This manual records ten experiments of the Computer Networks Laboratory. Experiments 1–4 are network simulations written in Tcl for the NS-2 simulator and analysed with AWK scripts and XGraph plots; Experiments 5 to 7 are data-link layer programs (bit stuffing, CRC and frame sorting) written in Python; Experiments 8 and 9 are client–server programs written in Python with TCP and UDP sockets, and Experiment 10 is a distance vector routing program written in C. Every experiment follows the same structure: aim, theory, topology or algorithm, program, execution, output and result; the viva voce questions are collected in Appendix A.

Software environment

Tool	Version	Purpose
Operating system	Ubuntu (Linux)	Lab workstation; all commands are run from a terminal
NS-2	2.35	Discrete-event network simulator, started with `ns script.tcl`
NAM	1.15	Network Animator; replays the `.nam` file written by NS-2
XGraph	4.38	Plots two-column (`x y`) data files
AWK	mawk / gawk	Post-processes the trace file (counts, throughput, congestion window)
Python	3.x	Programs of Experiments 5 to 9

Simulation workflow

An NS-2 simulation is described by a Tcl script. Running it produces two files: a *trace file* (`.tr`) that records every packet event, and a *NAM file* (`.nam`) used to animate the simulation. An AWK script reduces the trace file to the numbers or the data file that is finally plotted with XGraph.

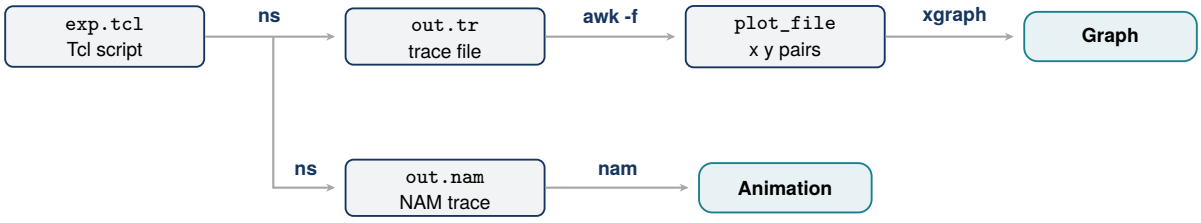


Figure 1. From Tcl script to results: the files and tools used in Experiments 1–4.

Reading the NS-2 trace file

Each line of the trace file describes one packet event. The first lines of the trace produced by Experiment 1 are shown below, followed by the meaning of the fields. AWK refers to the fields as `$1`, `$2`, … in this order.

out.tr (Experiment 1, first lines)

```
+ 0.5 0 1 cbr 500 ------- 0 0.0 2.0 0 0
- 0.5 0 1 cbr 500 ------- 0 0.0 2.0 0 0
+ 0.505 0 1 cbr 500 ------- 0 0.0 2.0 1 1
- 0.505 0 1 cbr 500 ------- 0 0.0 2.0 1 1
```

Field	Meaning
`$1`	event: `+` `-` `r` `d` (`h` on a LAN)
`$2`	time of the event (s)
`$3`	from node
`$4`	to node
`$5`	packet type (`cbr`, `tcp`, `ack`)
`$6`	packet size (bytes)

Field	Meaning
`$7`	flags
`$8`	flow id (`class_`)
`$9`	source address
`$10`	destination address
`$11`	sequence number
`$12`	unique packet id

The event symbols are: `+` packet *enqueued*, `-` packet *dequeued* (transmission starts), `r` packet *received* at the far end of the link, `d` packet *dropped*, and `h` a hop event on LAN links.

> **Units used in the scripts.** Bandwidth `1Mb` means 1 Mb/s, `512Kb` means 512 kb/s, delays are in milliseconds (`10ms`), packet sizes are in bytes and all times are in seconds. Throughput computed by the AWK scripts is expressed in kb/s.

Experiments at a glance

No.	Topic	Tools	Key result
1	Point-to-point network: packets dropped against bandwidth and queue size	NS-2, AWK, XGraph	502 drops at 0.25 Mb, none at 1 Mb
2	TCP and UDP traffic: packet counts and throughput	NS-2, AWK, XGraph	TCP 846 to 3582 records (1 to 2 Mb), UDP almost constant
3	Ethernet LAN: data rate and error rate	NS-2, AWK, XGraph	535 to 1369 kb/s for 0.5 to 3 Mb; errors lower the throughput
4	Congestion window of TCP Vegas and Reno	NS-2, AWK, XGraph	Vegas steady at about 22 packets, Reno grows to 30.7
5	Bit stuffing	Python	`1011111101` → `10111110101`
6	CRC error detection	Python	`1001` → codeword `1001110`
7	Frame sorting in buffers	Python	`2 3 1 4`→`1 2 3 4`
8	TCP sockets: file transfer client–server	Python	files received unchanged; missing file reported
9	UDP sockets: calculator client–server	Python	`12 + 5 = 17.0`; invalid input reported at the client
10	Distance vector routing	C (gcc)	A → C via B, cost 5

EXPERIMENT NO. 1 Date: 14/08/2026

Point-to-Point Network: Queue Size, Bandwidth and Packet Drops

Aim

> Simulate a three-node point-to-point network with duplex links between them. Set the queue size, vary the bandwidth and find the number of packets dropped.

Theory

Duplex links and the DropTail queue

In NS-2 a *duplex link* is a pair of simplex links, one for each direction, each described by a bandwidth, a propagation delay and a queue discipline. The command `$ns duplex-link $n0 $n1 1Mb 10ms DropTail` joins nodes 0 and 1 with a 1 Mb/s link of 10 ms delay. *DropTail* is a first-in-first-out queue with a finite buffer: when the buffer already holds `limit_` packets (50 by default, set explicitly with `queue-limit`), the newly arriving packet is discarded.

Traffic source

A `UDP` agent carries the traffic produced by a `CBR` (constant bit rate) application, and the `Null` agent at the destination simply absorbs the packets. With `packetSize_` = 500 bytes and `interval_` = 0.005 s the source offers

$$R = \frac{500 \times 8 \text{ bits}}{0.005 \text{ s}} = 800 \text{ kb/s}$$

from 0.5 s to 4.5 s, i.e. 800 packets in total.

Why packets are dropped

If the link bandwidth is larger than the offered load (1 Mb/s > 800 kb/s) each packet is transmitted as soon as it arrives and the queue stays empty. If the bandwidth is smaller, the queue grows at the rate $R - C$ (C = link capacity) until it is full, and every further arrival is dropped. The number of drops therefore decreases when the bandwidth or the queue size is increased. A drop is recorded in the trace file as a line beginning with `d`, and the AWK script simply counts these lines.

Network topology

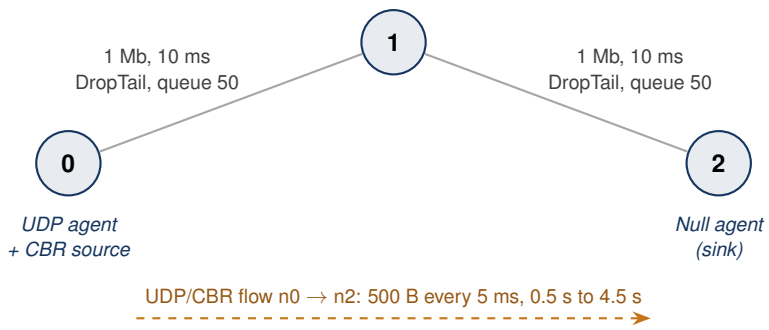


Figure 2. Three-node topology of Experiment 1 (node orientation as set with `duplex-link-op`).

Procedure

1. Create the simulator object and open the NAM file (`out.nam`) and the trace file (`out.tr`).
2. Write the `finish` procedure: flush the trace, close both files and start NAM.
3. Create three nodes `n0`, `n1` and `n2`.
4. Join them with duplex links (1 Mb, 10 ms, DropTail), set the orientation for NAM and set the queue limit of each link to 50 packets.
5. Attach a UDP agent to `n0` and a Null agent to `n2`, and connect them.
6. Attach a CBR application (500 bytes every 0.005 s) to the UDP agent; start it at 0.5 s and stop it at 4.5 s.
7. Schedule `finish` at 5.0 s, run the simulator with `ns exp1.tcl`, and count the `d` events with `awk -f exp1.awk out.tr`.
8. Repeat after changing the bandwidth of the links and, separately, the queue limit; tabulate the drops and plot them with XGraph.

Program

`exp1.tcl` — NS-2 simulation script

```
#gedit exp1.tcl
set ns [new Simulator]

#open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf

#open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf

#body of finish procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
}

#Create nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]

#Create links between nodes
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail

#set orientation
$ns duplex-link-op $n0 $n1 orient right-up
$ns duplex-link-op $n1 $n2 orient right-down

#set queue limit-default is 50 packets
$ns queue-limit $n0 $n1 50
$ns queue-limit $n1 $n2 50

#Create transport agent
#Sender
set udp0 [new Agent/UDP]
```

```
$ns attach-agent $n0 $udp0

#Receiver
set null0 [new Agent/Null]
$ns attach-agent $n2 $null0
$ns connect $udp0 $null0

#Create Application to generate traffic
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

#Start and stop generating traffic
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

$ns at 5.0 "finish"

$ns run
```

exp1.awk — counts the dropped packets

```
#gedit exp1.awk
BEGIN { count = 0; }
{
    if($1 == "d")
    {
        count++;
        printf("%s\t %s\n", $5, $11);
    }
}
END { printf(" The number of packets dropped =%d \n", count ); }
```

Execution and output

Terminal

```
$ ns exp1.tcl
$ awk -f exp1.awk out.tr
 The number of packets dropped =0
```

With the default bandwidth of 1 Mb the offered load (800 kb/s) fits in the link, so no packet is dropped. If the bandwidth is reduced (here to 0.5 Mb) the script prints the sequence number of every dropped packet and finally the total:

Terminal

```
$ awk -f exp1.awk out.tr        # bandwidth changed to 0.5Mb
cbr      131
cbr      134
cbr      136
...
 The number of packets dropped =252
```

The corresponding drop record in the trace file begins with the event symbol `d`:

out.tr (bandwidth 0.5 Mb)

```
d 1.155 0 1 cbr 500 ------- 0 0.0 2.0 131 131
```

Observations

The simulation was repeated for four bandwidths (queue limit 50) and for four queue limits (bandwidth 0.75 Mb). The counts were stored in the two data files `plot_graph_1` and `plot_graph_2` and plotted with XGraph.

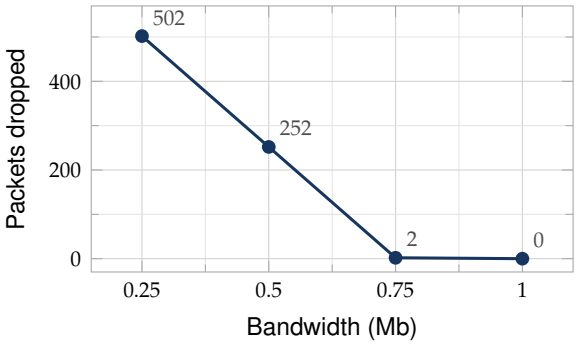
Bandwidth (Mb)	Packets dropped
0.25	502
0.5	252
0.75	2
1	0

Table 1: queue limit = 50 packets (`plot_graph_1`)

Queue limit	Packets dropped
40	12
30	22
20	32
10	42

Table 2: bandwidth = 0.75 Mb (`plot_graph_2`)

Graph 1: bandwidth vs. drop count



Graph 2: queue limit vs. drop count

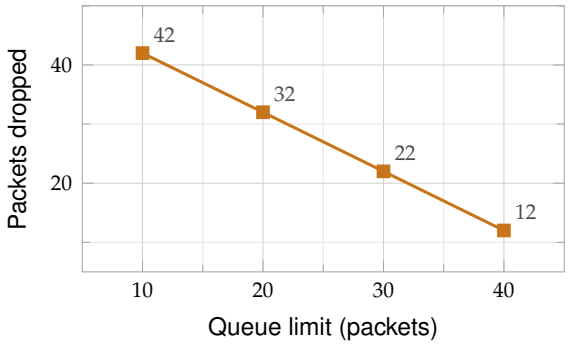


Figure 3. Packets dropped as a function of the link bandwidth and of the queue limit.

- At 1 Mb the link can carry the 800 kb/s offered load, so **no packet is dropped**.
- Reducing the bandwidth to 0.75, 0.5 and 0.25 Mb makes the service rate smaller than the arrival rate, and the drops rise to **2**, **252** and **502** packets.
- At 0.75 Mb a larger queue absorbs more of the excess: the drops fall from **42** (queue of 10 packets) to **12** (queue of 40 packets). A larger buffer reduces loss but increases the queuing delay.

Result. A three-node point-to-point network with duplex links was simulated in NS-2 and the number of dropped packets was determined from the trace file. The number of drops decreases as the link bandwidth or the queue size increases: from 502 packets at 0.25 Mb to none at 1 Mb, and from 42 packets (queue of 10) to 12 packets (queue of 40) at 0.75 Mb.

Viva voce questions for this experiment: see Appendix A, page 48.

EXPERIMENT NO. 2 Date: 21/08/2026

Four-Node Network with TCP and UDP Traffic: Packet Counts and Throughput

Aim

> Simulate a four-node point-to-point network and connect the links as follows: n0–n2, n1–n2 and n2–n3. Apply a TCP agent between n0–n3 and a UDP agent between n1–n3. Apply relevant applications over the TCP and UDP agents, change the parameters, determine the number of packets sent by TCP and UDP and analyse the throughput.

Theory

TCP and UDP in NS-2

UDP is connectionless and unreliable: it has no acknowledgements, no retransmission and no congestion control, so its sending rate is fixed by the application. Here the `CBR` application sends 500 bytes every 5 ms (800 kb/s). **TCP** is connection-oriented and reliable: the receiver acknowledges the data and the sender controls its *window* (slow start and congestion avoidance) so that it adapts to the capacity that is available. In NS-2 the sender is `Agent/TCP`, the receiver is `Agent/TCPSink` (it returns the acknowledgements) and `Application/FTP` supplies an unlimited stream of data to the TCP agent.

Shared bottleneck

Both flows meet at node 2 and share the link n2–n3. Because the UDP source never slows down, TCP obtains only the capacity that is left over, which is why the TCP traffic grows strongly when the link bandwidth is increased while the UDP traffic does not.

Counting packets and computing throughput

`exp2.awk` counts the trace records whose packet type (`$5`) is `cbr` (UDP) or `tcp`. A packet produces several records (enqueue, dequeue and receive on every link it crosses), so the counts are event counts that are used to compare the two protocols and the different bandwidths. `exp2_throughput.awk` adds up the bytes of all received packets (`r` events) and divides by the elapsed time:

$$\text{Throughput} = \frac{\sum \text{bytes of received packets}}{t_{\text{last receive}} - t_{\text{first enqueue}}} \times \frac{8}{1000} \ \text{kb/s}$$

Since a packet is counted at every link it crosses, this is the aggregate throughput of the whole network.

Network topology

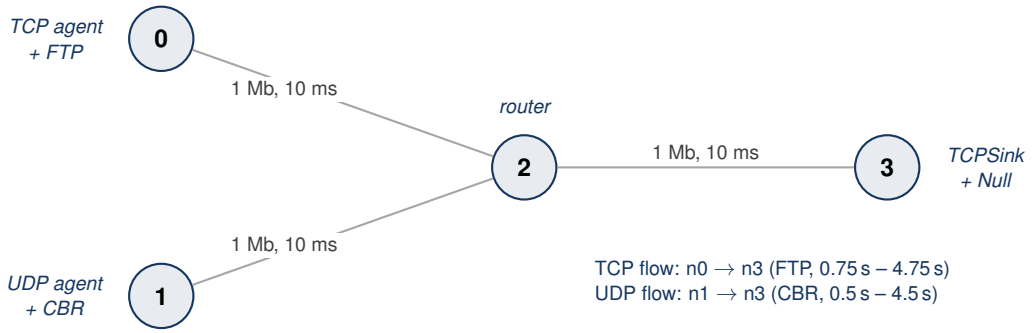


Figure 4. Four-node topology of Experiment 2. All links are 1 Mb, 10 ms, DropTail with a queue limit of 50 packets.

Procedure

1. Create the simulator, the NAM and trace files and the `finish` procedure.
2. Create four nodes, label them and join n0–n2, n1–n2 and n2–n3 with duplex links (1 Mb, 10 ms, DropTail); set the orientation and a queue limit of 50 packets.
3. Create a TCP agent on n0 and a TCPSink on n3, connect them and attach an FTP application to the TCP agent.
4. Create a UDP agent on n1 and a Null agent on n3, connect them and attach a CBR application (500 bytes, 0.005 s interval).
5. Start the CBR traffic at 0.5 s and stop it at 4.5 s; start the FTP traffic at 0.75 s and stop it at 4.75 s; call `finish` at 5.0 s.
6. Run `ns exp2.tcl`; count the TCP and UDP records with `exp2.awk`.
7. Repeat for other link bandwidths and record the counts in the files `plot_tcp` and `plot_udp`; plot them with XGraph.
8. Generate the throughput data with `exp2_throughput.awk` and plot it against time.

Program

`exp2.tcl` — NS-2 simulation script

```
set ns [new Simulator]

#open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf

#open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf

#body of finish procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
```

```
}

#Create 4 nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

#set label
$n0 label "TCP"
$n1 label "UDP"
$n2 label "Receiver"

#Create links between nodes
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail

#set orientation
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right

#set the queue limit-default is 50 packets
$ns queue-limit $n0 $n2 50
$ns queue-limit $n1 $n2 50
$ns queue-limit $n2 $n3 50

#Create TCP agent b/w node 0 and node 3
#Sender
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0

#Receiver
set sink0 [new Agent/TCPSink]
$ns attach-agent $n3 $sink0
$ns connect $tcp0 $sink0

#Create FTP Application for TCP Agent
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

#Specify TCP packet size
Agent/TCP set packetSize_ 1000

#Create UDP Agent b/w node1 and node3
#Sender
set udp0 [new Agent/UDP]
$ns attach-agent $n1 $udp0

#Receiver
set null0 [new Agent/Null]
$ns attach-agent $n3 $null0
$ns connect $udp0 $null0

#Create CBR Application for UDP Agent
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

#Start and Stop FTP Traffic
$ns at 0.75 "$ftp0 start"
$ns at 4.75 "$ftp0 stop"

#Start and stop CBR traffic
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

#Stop the simulations
$ns at 5.0 "finish"
```

```
$ns run
```

exp2.awk — packet counts of TCP and UDP

```
#gedit exp2.awk
BEGIN {
    tcp_count = 0;
    udp_count = 0;
}
{
    if($5 == "cbr")
        udp_count++;
    else if($5 == "tcp")
        tcp_count++;
}
END {
    printf("The number of packets by TCP =%d\n", tcp_count);
    printf("The number of packets by UDP =%d\n", udp_count);
}
```

exp2_throughput.awk — throughput against time

```
#gedit exp2_throughput.awk
BEGIN {
    sSize = 0;
    startTime = 5.0;
    stopTime = 0.1;
    Tput = 0;
}
{
    event = $1;
    time = $2;
    size = $6;
    if(event == "+")
    {
        if (time < startTime) {
            startTime = time;
        }
    }
    if(event == "r")
    {
        if(time > stopTime)
        {
            stopTime = time;
        }
        sSize += size;
    }
    Tput = (sSize / (stopTime - startTime)) * (8/1000);
    printf("%f\t %.2f\n", time, Tput);
}
END {
}
```

Execution and output

Terminal

```
$ ns exp2.tcl
$ awk -f exp2.awk out.tr
The number of packets by TCP =846
The number of packets by UDP =4786
```

The throughput data is written to a file and plotted; the file has one `time throughput` pair per line. The first values are (negative) zero because nothing has been received yet.

Terminal

```
$ awk -f exp2_throughput.awk out.tr > plot_throughput
$ xgraph plot_throughput
$ head -3 plot_throughput; tail -2 plot_throughput
0.500000          -0.00
0.500000          -0.00
0.505000          -0.00
...
4.800920          2017.06
4.809240          2013.24
```

Observations

The simulation was repeated for five link bandwidths. The counts are the values stored in `plot_tcp` (TCP) and `plot_udp` (UDP).

Bandwidth (Mb)	TCP records	UDP records
1	846	4786
1.25	1509	4791
1.5	2071	4805
1.75	2892	4806
2	3582	4806

Table 3: number of TCP and UDP packet records against link bandwidth

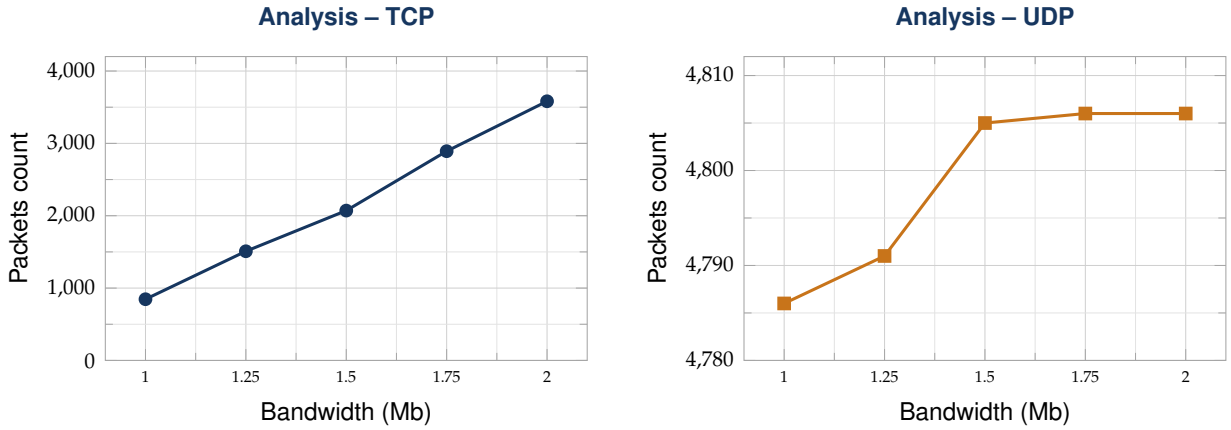


Figure 5. TCP and UDP packet counts for different link bandwidths (`xgraph plot_tcp` and `xgraph plot_udp`). Note the very different vertical scales.

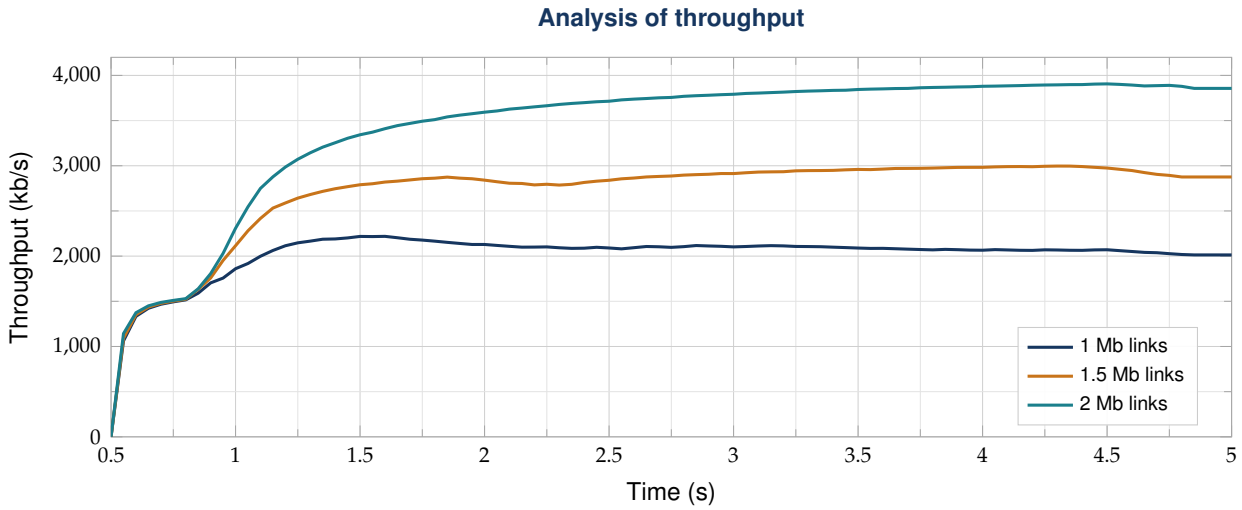


Figure 6. Aggregate throughput against time (`xgraph plot_throughput`) for three link bandwidths; each curve was produced by re-running `exp2.tcl` with the bandwidth changed.

- The **TCP** count grows from 846 to 3582 (about four times) when the bandwidth is doubled from 1 Mb to 2 Mb, because TCP uses the capacity that the UDP flow leaves free.
- The **UDP** count stays almost constant (4786 to 4806): the CBR source sends at a fixed rate irrespective of the available bandwidth.
- The throughput rises quickly once the sources start at 0.5 s and then levels off. At the end of the run it is about 2013 kb/s (1 Mb), 2876 kb/s (1.5 Mb) and 3856 kb/s (2 Mb): a higher bandwidth gives a higher throughput.

Result. A four-node network carrying a TCP flow (n0 → n3) and a UDP flow (n1 → n3) was simulated in NS-2. The TCP traffic increased with the link bandwidth (846 to 3582 records for 1 Mb to 2 Mb), the UDP traffic remained nearly constant, and the network throughput increased with the bandwidth.

Viva voce questions for this experiment: see Appendix A, page 48.

EXPERIMENT NO. 3 Date: 28/08/2026

Ethernet LAN: Throughput versus Data Rate and Error Rate

Aim

> Simulate an Ethernet LAN using N nodes (6–10). Change the error rate and the data rate and compare the throughput.

Theory

Ethernet and CSMA/CD

In an Ethernet LAN (IEEE 802.3) all stations share one transmission medium. A station first *senses* the channel and transmits only when it is idle (carrier sense, multiple access). While transmitting it keeps listening; if another station transmits at the same time a *collision* is detected, both stations stop, and each waits for a random time before trying again. The waiting time follows *binary exponential backoff*: after the i-th collision a station waits a random number of slot times between 0 and $2^{\min(i,10)} - 1$, and gives up after 16 attempts.

LANs in NS-2

The command

```
$ns make-lan "$n1 $n2 $n3 $n4 $n5 $n6" 2Mb 40ms LL Queue/DropTail Mac/802_3
```

creates a shared LAN of the listed nodes with a bandwidth of 2 Mb/s and a delay of 40 ms. Every node gets a link layer (`LL`), an interface queue (`Queue/DropTail`) and an Ethernet MAC (`Mac/802_3`) that implements CSMA/CD. In this experiment seven nodes are used: n0 is joined to the LAN node n1 by a point-to-point link (3 Mb, 20 ms), and n1–n6 form the LAN. A TCP source with an FTP application on n0 sends to a TCP sink on n6.

Data rate and error rate

The *data rate* is the bandwidth given to `make-lan`. An *error rate* is introduced with an NS-2 `ErrorModel` object that discards each packet crossing a link with a given probability. TCP treats every loss as a sign of congestion and reduces its window (or waits for a timeout), so both a lower data rate and a higher error rate reduce the throughput. The throughput is computed by `exp3.awk` with the same formula as in Experiment 2.

Network topology

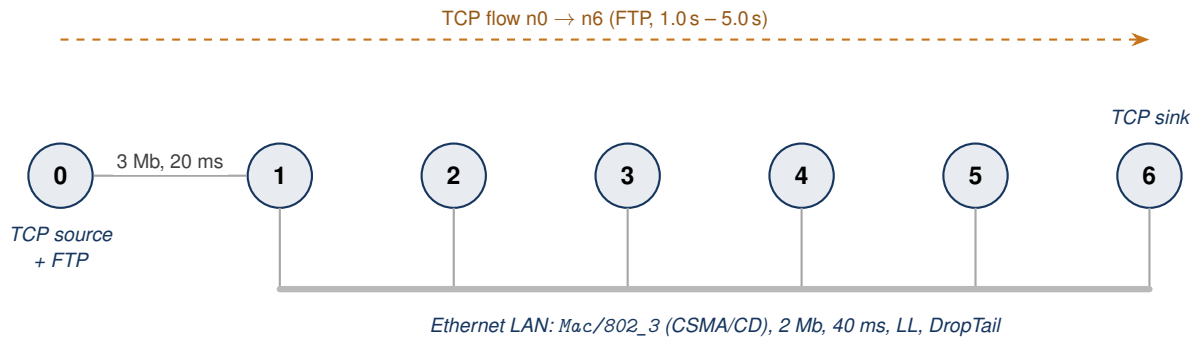


Figure 7. Topology of Experiment 3: a point-to-point link followed by a six-node Ethernet LAN.

Procedure

1. Create the simulator, the NAM file `exp3.nam`, the trace file `exp3.tr` and a `finish` procedure that also runs `exp3.awk` to produce `exp3.graph`.
2. Create seven nodes, label n0 (TCP source) and n6 (TCP sink), and set the colours of the flow and of the nodes.
3. Join n0 and n1 with a 3 Mb, 20 ms duplex link and build the LAN of n1–n6 with `make-lan` (2 Mb, 40 ms, `Mac/802_3`).
4. Create a TCP agent on n0 and a TCPSink on n6, connect them, and attach an FTP application.
5. Start the FTP traffic at 1.0 s, stop it at 5.0 s and call `finish` at 5.5 s.
6. Run `ns exp3.tcl` and plot `exp3.graph` with XGraph.
7. **Change the data rate:** edit the LAN bandwidth (`2Mb`) and run again; keep a copy of `exp3.graph` for every rate and plot them together.
8. **Change the error rate:** insert the `ErrorModel` lines given below, set `rate_` to the required value, run again and compare the throughput.

Program

exp3.tcl — NS-2 simulation script

```
set ns [new Simulator]
set namfile [open exp3.nam w]
$ns namtrace-all $namfile
set tracefile [open exp3.tr w]
$ns trace-all $tracefile

proc finish {} {
    global ns namfile tracefile
    $ns flush-trace
    close $namfile
    close $tracefile
    exec nam exp3.nam &
    exec awk -f exp3.awk exp3.tr > exp3.graph &
    exit 0
}

#Create 6 nodes
```

```
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]

#label source and sink nodes
$n0 label "TCPSource"
$n6 label "TCPSink"

#color to the flow
$ns color 1 red

#color to the nodes
$n0 color blue
$n6 color orange

$ns duplex-link $n0 $n1 3Mb 20ms DropTail
$ns make-lan "$n1 $n2 $n3 $n4 $n5 $n6" 2Mb 40ms LL Queue/DropTail Mac/802_3

$ns duplex-link-op $n0 $n1 orient right
set tcp [new Agent/TCP]
$ns attach-agent $n0 $tcp
$tcp set class_ 1

set tcpsink [new Agent/TCPSink]
$ns attach-agent $n6 $tcpsink
$ns connect $tcp $tcpsink

set ftp [new Application/FTP]
$ftp attach-agent $tcp

$ns at 1.0 "$ftp start"
$ns at 5.0 "$ftp stop"
$ns at 5.5 "finish"
$ns run
```

exp3.awk — throughput against time

```
#gedit exp3.awk
BEGIN {
    sSize = 0;
    startTime = 5.0;
    stopTime = 0.1;
    Tput = 0;
}
{
    event = $1;
    time = $2;
    size = $6;
    if (event == "+")
    {
        if (time < startTime)
        {
            startTime = time;
        }
    }
    if (event == "r")
    {
        if (time > stopTime)
        {
            stopTime = time;
        }
        sSize += size;
    }
    Tput = (sSize/(stopTime-startTime)) * (8/1000);
    printf("%f\t %.2f\n", time, Tput);
}
END {
```

```
}
```

Lines added to introduce an error rate

The following lines are inserted into `exp3.tcl` directly after the command that creates the n0–n1 link. `rate_ 0.01` drops 1 % of the packets that cross the link from n0 to n1.

error model (added after the n0-n1 duplex-link command)

```
#Error model on the link n0 -> n1
set em [new ErrorModel]
$em unit pkt
$em set rate_ 0.01
$em ranvar [new RandomVariable/Uniform]
$em drop-target [new Agent/Null]
$ns lossmodel $em $n0 $n1
```

Execution and output

Terminal

```
$ ns exp3.tcl
$ xgraph exp3.graph
$ head -3 exp3.graph; tail -2 exp3.graph
1.000000          -0.00
1.000000          -0.00
1.020107          15.91
...
5.202373          1328.39
5.206637          1327.12
```

`exp3.graph` is written by the `awk` command inside the `finish` procedure. When the `ns` program prints `warning: no class variable LanRouter::debug_` this is normal for `make-lan` and can be ignored.

Observations

Effect of the data rate

The LAN bandwidth was changed from 2 Mb to 0.5 Mb, 1 Mb and 3 Mb while everything else was kept constant.

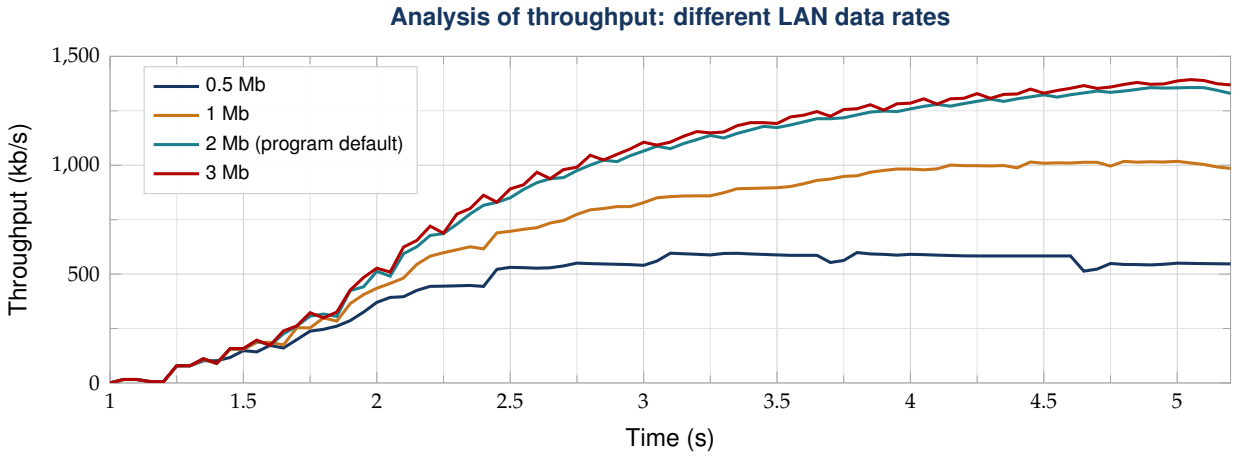


Figure 8. Throughput against time for four LAN data rates (`xgraph exp3.graph` for each run).

Effect of the error rate

The `ErrorModel` of the previous page was inserted with `rate_` set to 0, 0.001, 0.005 and 0.01 (error rates of 0 %, 0.1 %, 0.5 % and 1 %); the LAN data rate was 2 Mb.

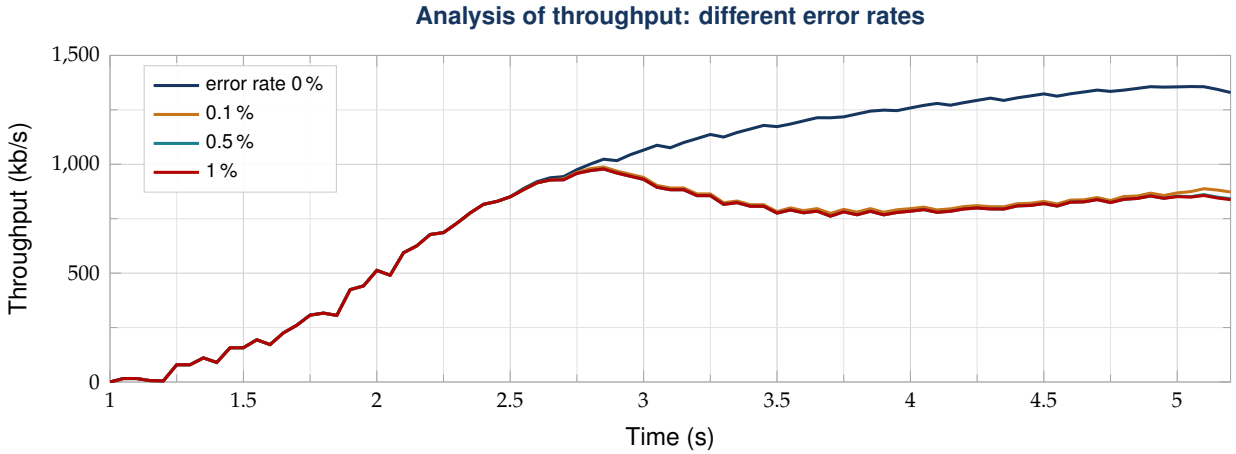


Figure 9. Throughput against time for four error rates on the link n0–n1.

LAN data rate	Throughput at 5.2 s
0.5 Mb	535 kb/s
1 Mb	985 kb/s
2 Mb	1327 kb/s
3 Mb	1369 kb/s

Table 4: error rate = 0

Error rate	Throughput at 5.2 s
0 %	1327 kb/s
0.1 %	871 kb/s
0.5 %	842 kb/s
1 %	805 kb/s

Table 5: LAN data rate = 2 Mb

- **Data rate.** The throughput rises almost in proportion to the data rate from 0.5 Mb to 1 Mb (535 to 985 kb/s). Beyond 2 Mb the gain is small (1327 to 1369 kb/s): the LAN is then no longer the bottleneck and the growth of the TCP window over a path with a round-trip delay of at least 120 ms limits the rate.
- **Error rate.** Even a small error rate lowers the throughput clearly, from 1327 kb/s without errors to 871 kb/s at 0.1 % and 805 kb/s at 1 %, because each lost packet makes TCP reduce its window or wait for a retransmission timeout. The exact values depend on which packets are lost, since the error model drops packets at random.

Result. An Ethernet LAN of six nodes (n1–n6) with a TCP source on n0 and a sink on n6 was simulated in NS-2. The throughput increased with the data rate until the TCP flow itself became the limit (535 kb/s at 0.5 Mb to 1369 kb/s at 3 Mb), and it decreased when an error rate was introduced (1327 kb/s without errors, 805 kb/s at 1 % errors).

Viva voce questions for this experiment: see Appendix A, page 48.

EXPERIMENT NO. 4 Date: 04/09/2026

Ethernet LAN with Multiple Traffic Flows: Congestion Window of TCP Vegas and Reno

Aim

> Simulate an Ethernet LAN using N nodes and set multiple traffic nodes, and determine the collisions across the different nodes. Also plot the congestion window for different source/destination pairs.

Theory

Congestion window

A TCP sender may have only a limited amount of unacknowledged data in the network. This limit is the *congestion window* (`cwnd_`), measured here in packets. In *slow start* the window grows by one packet per acknowledgement, i.e. it doubles every round-trip time; in *congestion avoidance* it grows by about one packet per round-trip time. In NS-2 the number of packets actually in flight is also bounded by the window parameter `window_` (20 packets by default), while the variable `cwnd_` itself keeps changing according to the TCP variant.

TCP Reno and TCP Vegas

Reno is loss based: it keeps increasing the window until a packet is lost and then halves it (fast retransmit and fast recovery). **Vegas** is delay based: it compares the expected rate (window divided by the smallest round-trip time) with the actual rate and keeps the difference between two thresholds, so that only a few packets are queued in the network. The window therefore settles at a steady value instead of growing until a loss occurs.

Shared LAN and collisions

Two TCP flows, n1 $\rightarrow$ n7 (Vegas) and n2 $\rightarrow$ n8 (Reno), cross the router n0 and then the Ethernet LAN formed by n3–n8 (512 kb/s, 40 ms, `Mac/802_3`). On the shared LAN the data packets and the acknowledgements of both flows compete for the medium; CSMA/CD detects collisions and resolves them by random backoff, as described in Experiment 3. Colours and shapes (`color`, `shape`) are used only to make the flows and the nodes n7 (box) and n8 (hexagon) recognisable in NAM. The congestion window of a connection is written to a file with `$tcp1 attach $tfile1` and `$tcp1 trace cwnd_`.

Network topology

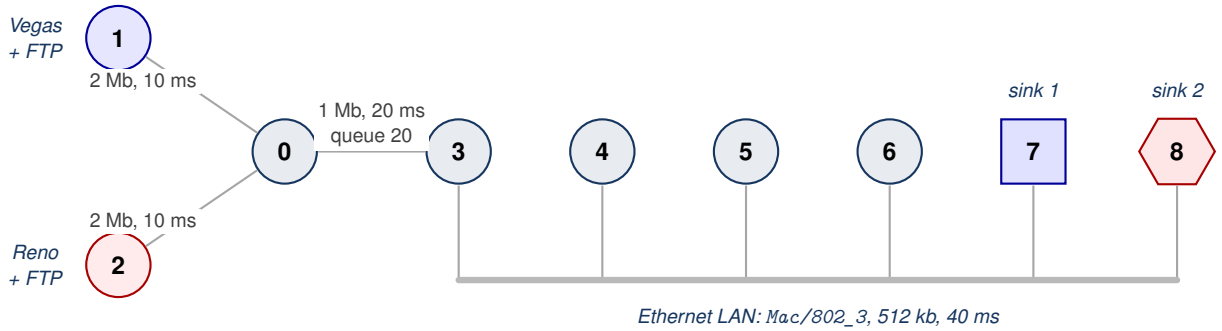


Figure 10. Topology of Experiment 4. Flow 1 (blue): TCP Vegas n1 → n7; flow 2 (red): TCP Reno n2 → n8.

Flow	TCP variant	Path	Starts	`class_`	cwnd file	Plot file
1	Vegas (`Agent/TCP/Vegas`)	n1 → n7	0.5 s	1 (blue)	`cwnd1.tr`	`tcp1`
2	Reno (`Agent/TCP/Reno`)	n2 → n8	1.0 s	2 (red)	`cwnd2.tr`	`tcp2`

Table 6: the two traffic flows of Experiment 4 (both stop at 5.0 s; packet size 55 bytes)

Procedure

1. Create the simulator, the NAM file `exp4.nam`, the trace file `exp4.tr`, and a `finish` procedure that closes the congestion-window files and extracts `tcp1` and `tcp2` with `exp4.awk`.
2. Create nine nodes n0–n8; set the flow colours (1 blue, 2 red) and the shapes and colours of n7 and n8.
3. Join n1–n0 and n2–n0 (2 Mb, 10 ms) and n0–n3 (1 Mb, 20 ms), build the LAN n3–n8 with `make-lan` (512 Kb, 40 ms, `Mac/802_3`) and set the queue limit of n0–n3 to 20 packets.
4. Create the TCP Vegas agent on n1 with its sink on n7 and an FTP source; create the TCP Reno agent on n2 with its sink on n8 and an FTP source.
5. Open `cwnd1.tr` and `cwnd2.tr`, attach them to the agents and trace `cwnd_`.
6. Start FTP 1 at 0.5 s and FTP 2 at 1.0 s, stop both at 5.0 s and call `finish` at 5.5 s.
7. Run `ns exp4.tcl` and plot the two files with `xgraph tcp1` and `xgraph tcp2`.

Program

`exp4.tcl` — NS-2 simulation script

```
set ns [new Simulator]
set namfile [open exp4.nam w]
$ns namtrace-all $namfile

set tracefile [open exp4.tr w]
$ns trace-all $tracefile

proc finish {} {
    global ns namfile tracefile tfile1 tfile2
    $ns flush-trace
    close $namfile
    close $tracefile
    close $tfile1
```

```
    close $tfile2
    exec nam exp4.nam &
    exec awk -f exp4.awk cwnd1.tr > tcp1 &
    exec awk -f exp4.awk cwnd2.tr > tcp2 &
    exit 0
}

#Create the nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]
set n7 [$ns node]
set n8 [$ns node]

#color to packet flow
$ns color 1 Blue
$ns color 2 Red

#set shape and color to node 7 and node 8
$n7 shape Box
$n7 color Blue
$n8 shape hexagon
$n8 color Red

$ns duplex-link $n1 $n0 2Mb 10ms DropTail
$ns duplex-link $n2 $n0 2Mb 10ms DropTail
$ns duplex-link $n0 $n3 1Mb 20ms DropTail

#Create LAN
$ns make-lan "$n3 $n4 $n5 $n6 $n7 $n8" 512Kb 40ms LL Queue/DropTail Mac/802_3

#Set Orientation
$ns duplex-link-op $n1 $n0 orient right-down
$ns duplex-link-op $n2 $n0 orient right-up
$ns duplex-link-op $n0 $n3 orient right
$ns queue-limit $n0 $n3 20

#set the transport protocol
set tcp1 [new Agent/TCP/Vegas]
$ns attach-agent $n1 $tcp1

set sink1 [new Agent/TCPSink]
$ns attach-agent $n7 $sink1

#set the application
set ftp1 [new Application/FTP]
$ftp1 attach-agent $tcp1

$ns connect $tcp1 $sink1
$tcp1 set class_ 1
$tcp1 set packetSize_ 55

set tfile1 [open cwnd1.tr w]
$tcp1 attach $tfile1
$tcp1 trace cwnd_

set tcp2 [new Agent/TCP/Reno]
$ns attach-agent $n2 $tcp2

set sink2 [new Agent/TCPSink]
$ns attach-agent $n8 $sink2

set ftp2 [new Application/FTP]
$ftp2 attach-agent $tcp2

$ns connect $tcp2 $sink2
$tcp2 set class_ 2
```

```
$tcp2 set packetSize_ 55

set tfile2 [open cwnd2.tr w]
$tcp2 attach $tfile2
$tcp2 trace cwnd_

$ns at 0.5 "$ftp1 start"
$ns at 1.0 "$ftp2 start"
$ns at 5.0 "$ftp2 stop"
$ns at 5.0 "$ftp1 stop"
$ns at 5.5 "finish"
$ns run
```

exp4.awk — extracts time and congestion window

```
# exp4.awk
BEGIN {
}
{
    if($6 == "cwnd_")
    {
        printf("%f\t %f\n", $1, $7);
    }
}
END {
}
```

Execution and output

Terminal

```
$ ns exp4.tcl
$ xgraph tcp1
$ xgraph tcp2
```

The files written by NS-2 contain one line per change of the congestion window: time, source node, source port, destination node, destination port, the traced variable and its value. The AWK script keeps lines whose sixth field is `cwnd_` and prints the time and the value.

cwnd1.tr (first line) and tcp1 (extracted data)

```
0.00000  1  0  7  0  cwnd_ 1.000
0.94645  1  0  7  0  cwnd_ 2.000
$ head -4 tcp1; tail -2 tcp1
0.000000          1.000000
0.946450          2.000000
0.946450          3.000000
1.392900          4.000000
...
5.483430          1.000000
5.483430          2.000000
```

Observations

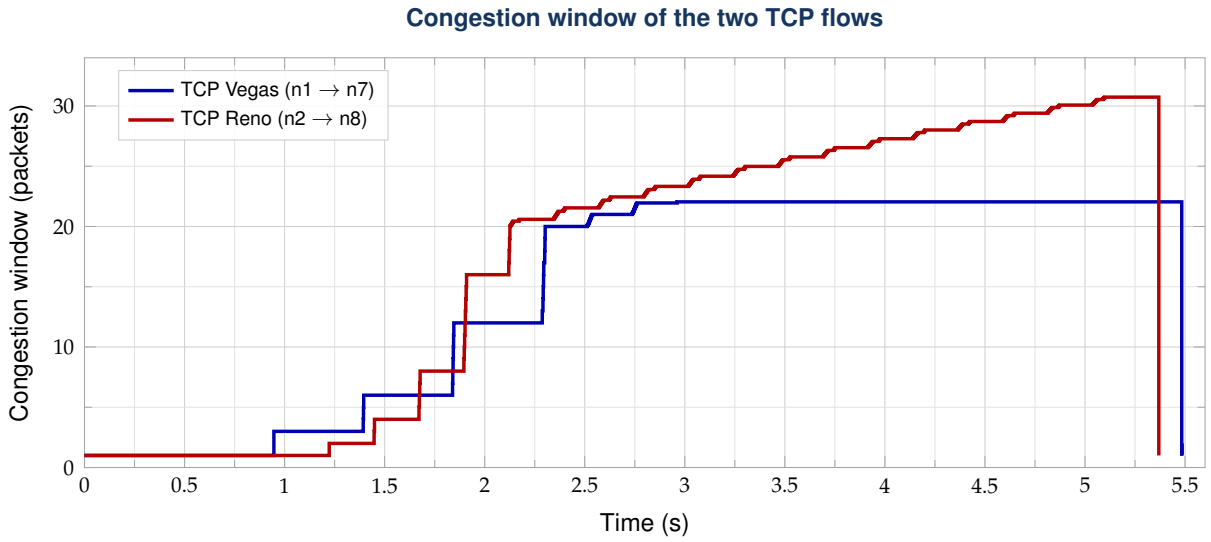


Figure 11. Congestion window against time (`xgraph tcp1` and `xgraph tcp2` drawn in one plot).

- Both connections begin in **slow start**: the window grows quickly from 1 packet (Vegas from 0.5 s, Reno from 1.0 s).
- **Vegas** leaves slow start at 20 packets (2.3 s, the NS-2 default threshold), grows slowly to 22.0 packets at $t = 2.96$ s and then keeps this value: no further change was logged until the sources stopped. Its delay-based control has found a steady operating point.
- **Reno** leaves slow start at about 2.1 s (also at 20 packets) and then grows linearly (congestion avoidance), reaching 30.7 packets at $t = 5.10$ s. It never reduces the window because no packet is lost: the trace file contains no `d` (drop) records.
- Both curves end with a reset of the window to 1 packet shortly after the FTP sources stop at 5.0 s.
- The two flows share the LAN without any dropped packet: contention for the medium is resolved by CSMA/CD backoff and every packet is delivered.

Result. An Ethernet LAN with two simultaneous TCP flows was simulated in NS-2, and the congestion windows of TCP Vegas (n1 → n7) and TCP Reno (n2 → n8) were extracted from the trace and plotted. Vegas settled at a constant window of about 22 packets, whereas Reno kept increasing its window (30.7 packets at 5.1 s); no packet was dropped.

Viva voce questions for this experiment: see Appendix A, page 49.

EXPERIMENT NO. 5 Date: 11/09/2026

Bit Stuffing in the Data-Link Layer

Aim

> Write a program to implement the bit stuffing (or byte stuffing) concept in the data-link layer.

Theory

Framing and the need for stuffing

The data-link layer groups the bits it receives from the network layer into *frames*. One way of marking the start and the end of a frame is a special *flag* pattern; HDLC uses the bit pattern `01111110`. If the data itself contained this pattern, the receiver would mistake it for the end of the frame. *Stuffing* makes the data transparent by making sure that the flag pattern can never occur inside the data.

Bit stuffing

- **Sender:** whenever five consecutive 1s have been sent, a 0 is inserted immediately after them, whatever the next data bit is.
- **Receiver:** whenever five consecutive 1s are received, the next bit (a stuffed 0) is removed.

Hence six consecutive 1s can only occur inside a flag. In the worst case, when the data consists only of 1s, one extra bit is added for every five data bits (an overhead of 20 %).

Byte stuffing

Character-oriented protocols (for example PPP) use a flag *byte* (`0x7E`) and an escape byte (`0x7D`). Every flag or escape byte inside the data is preceded by an escape byte (PPP additionally XORs the data byte with `0x20`). The program of this experiment implements **bit stuffing**.

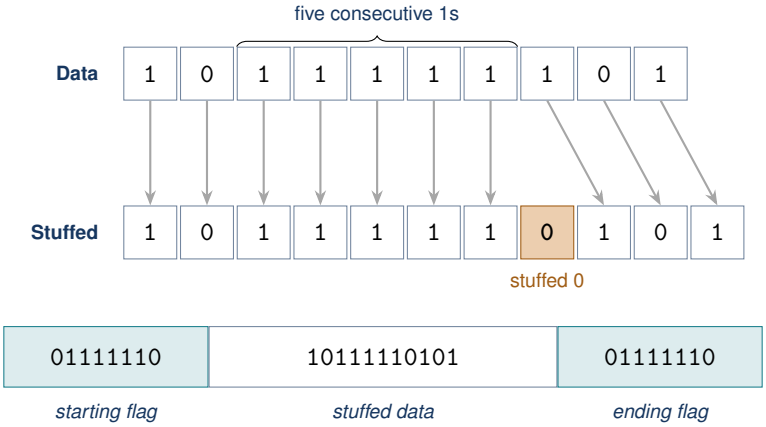


Figure 12. Bit stuffing of the data `1011111101` and the resulting frame (the stuffed 0 is highlighted).

Algorithm

Bit stuffing (sender)

1. Set `count = 0` and let `stuffed_data` be empty.
2. For each data bit, append the bit to `stuffed_data`.
3. If the bit is `1`, add 1 to `count`; when `count` reaches 5, append a `0` and set `count = 0`.
4. If the bit is `0`, set `count = 0`.
5. Return `stuffed_data`.

Bit destuffing (receiver)

1. Set `count = 0`, `skip_next = False` and let `destuffed_data` be empty.
2. For each received bit: if `skip_next` is set, clear it and ignore the bit (it is the stuffed 0).
3. Otherwise append the bit; if it is `1`, add 1 to `count`; when `count` reaches 5, set `skip_next` and `count = 0`.
4. If the bit is `0`, set `count = 0`.
5. Return `destuffed_data`.

Program

exp5.py — bit stuffing and destuffing

```
def bit_stuffing(data):
    stuffed_data = ""
    count = 0
    for bit in data:
        stuffed_data += bit
        if bit == '1':
            count += 1
            if count == 5:
                stuffed_data += '0'
                count = 0
        else:
            count = 0
    return stuffed_data

def bit_destuffing(stuffed_data):
    destuffed_data = ""
    count = 0
    skip_next = False
    for bit in stuffed_data:
        if skip_next:
            skip_next = False
            continue
        destuffed_data += bit
        if bit == '1':
            count += 1
            if count == 5:
                skip_next = True
                count = 0
        else:
            count = 0
    return destuffed_data

data = input("Enter the input data:")
print("Original data:", data)
stuffed = bit_stuffing(data)
print("Bit Stuffed data:", stuffed)
destuffed = bit_destuffing(stuffed)
print("Bit Destuffed data:", destuffed)
```

Execution and output

Terminal

```
$ python3 exp5.py
Enter the input data:1011111101
Original data: 1011111101
Bit Stuffed data: 10111110101
Bit Destuffed data: 1011111101
```

Further test runs, including the special cases of five and ten consecutive 1s and of data without any run of 1s:

Input data	Bit stuffed data	Bit destuffed data
1011111101	10111110101	1011111101
11111	111110	11111
1111111111	111110111110	1111111111
0000	0000	0000

Table 7: results of the program for four input strings

Observations

- A `0` is inserted after every run of five consecutive 1s, even when the run ends the data (`11111` → `111110`); data without such a run is unchanged (`0000`).
- The destuffed data is identical to the original data in every case, so the transmission is transparent.
- For the all-ones input of ten bits two extra bits are added, which is the 20 % worst-case overhead.

Result. A Python program was written to perform bit stuffing at the sender and bit destuffing at the receiver. The input `1011111101` was stuffed to `10111110101` and destuffed back to the original data.

Viva voce questions for this experiment: see Appendix A, page 49.

EXPERIMENT NO. 6 Date: 09/10/2026

Error Detection Using CRC (CRC-CCITT)

Aim

> Write a program for an error-detecting code using CRC-CCITT (16-bit).

Theory

Cyclic redundancy check

A CRC treats a bit string as a polynomial with binary coefficients; for example `1001` is $x^3 + 1$. Sender and receiver agree on a *generator polynomial* $G(x)$ of degree r (that is, $r + 1$ bits). All arithmetic is modulo 2: addition and subtraction are both the exclusive OR (XOR) and there are no carries.

1. **Sender:** append r zeros to the k message bits, giving $M(x)\,x^r$.
2. Divide $M(x)\,x^r$ by $G(x)$ (modulo 2) and keep the r-bit remainder $R(x)$.
3. Transmit $T(x) = M(x)\,x^r + R(x)$; this codeword is exactly divisible by $G(x)$.
4. **Receiver:** divide the received bits by $G(x)$. A remainder of zero means that no error was detected; a non-zero remainder means that the frame is corrupted.

CRC-CCITT

The CRC-CCITT generator is $G(x) = x^{16} + x^{12} + x^5 + 1$, i.e. the 17-bit pattern `1 0001 0000 0010 0001` (hexadecimal `0x11021`), so $r = 16$ check bits are appended. A 16-bit CRC detects all single-bit and double-bit errors, all errors with an odd number of bits, all burst errors of length 16 or less, and 99.997 % of 17-bit bursts. An error pattern goes undetected only if it is itself divisible by $G(x)$.

The program of this experiment keeps the generator in the list `div`. To keep the hand calculation short it is set to `[1, 0, 1, 1]`, i.e. $G(x) = x^3 + x + 1$ and $r = 3$; the program works unchanged for any generator, and the replacement that gives the 16-bit CRC-CCITT is shown at the end of this experiment.

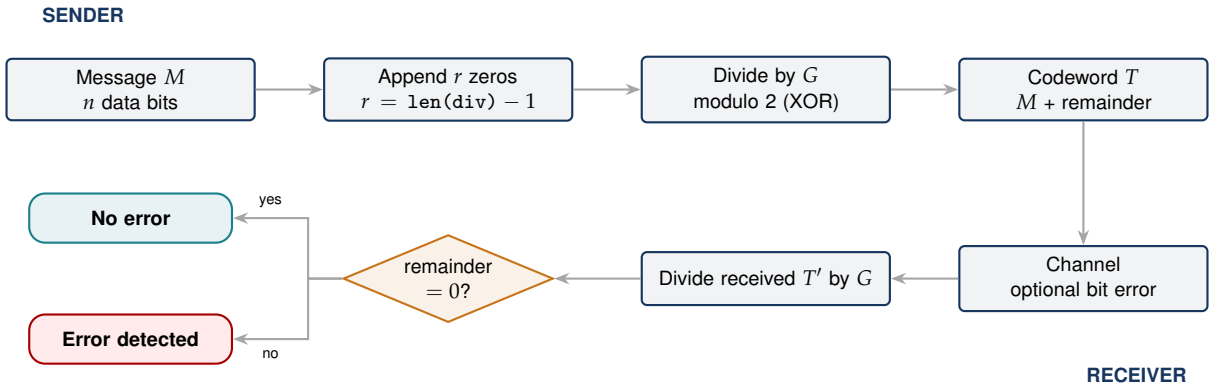


Figure 13. Principle of CRC error detection.

Worked example

Message `1001` and generator `1011` ($r = 3$). Three zeros are appended and the division is carried out by XOR-ing the generator under every leading 1:

Sender: 1001000 divided by 1011

```
       1 0 0 1 0 0 0
XOR    1 0 1 1
       -------------
       0 0 1 0 0 0 0
XOR        1 0 1 1
       -------------
       0 0 0 0 1 1 0      remainder = 110   ->   codeword = 1001 110
```

The receiver divides the received codeword `1001110` in the same way and obtains the remainder `000`. If the first bit is corrupted (`0001110`) the remainder is `101`:

Receiver: error-free codeword and corrupted codeword

```
received 1001110                          received 0001110 (bit 1 flipped)
       1 0 0 1 1 1 0                             0 0 0 1 1 1 0
XOR    1 0 1 1                                   (bits 0-2 are 0: no XOR)
       -------------                      XOR          1 0 1 1
       0 0 1 0 1 1 0                             -------------
XOR        1 0 1 1                               0 0 0 0 1 0 1
       -------------
       0 0 0 0 0 0 0      remainder 000          remainder 101 -> error
```

Algorithm

1. Read the number of data bits n and the bits; let m be the length of the generator `div`. If $n < m$ the data is invalid.
2. Copy the data to `cdata` and append $m - 1$ zeros (initial codeword).
3. For $i = 0, \ldots, n - 1$: if `cdata[i]` is 1, XOR the m generator bits into `cdata[i..i+m-1]`.
4. The last $m - 1$ bits of `cdata` are the remainder; append them to the data to form the codeword and display it.
5. Optionally flip the bit at a position chosen by the user (error introduction) and display the received message.
6. Divide the received message by the generator again; if every bit of the result is 0 print *No Error*, otherwise print *Error!*.

Program

exp6.py — CRC generation and error detection

```python
div = [1, 0, 1, 1]
cdata = []
n = int(input("Enter the number of bits:"))
data = [int(input(f"Enter bit {i + 1}:")) for i in range(n)]
m = len(div)

if n < m:
    print("Invalid data")
else:
    cdata = data.copy()
    for i in range(m - 1):
        cdata.append(0)
    print("Initial Codeword:", cdata)
```

```
    for i in range(n):
        if cdata[i] == 1:
            for j in range(m):
                cdata[i + j] ^= div[j]

    for i in range(n, n + m - 1):
        data.append(cdata[i])
    print("Codeword to be transmitted is:", data)
    ch = input("Introduce Error - Y -> YES N->NO:")
    if ch.upper() == 'Y':
        p = int(input("Enter the position to change bit:"))
        if 1 <= p <= len(data):
            data[p-1] ^= 1
        else:
            print("Invalid Position!")
    print("The message received is:", data)
    for i in range(n):
        if data[i] == 1:
            for j in range(m):
                data[i + j] ^= div[j]
    flag = True
    for i in data:
        if i == 1:
            flag = False
            break
    if (flag):
        print("No Error\nSuccessfully Transferred")
    else:
        print("Error!")
```

Execution and output

Run 1: no error introduced

Terminal

```
$ python3 exp6.py
Enter the number of bits:4
Enter bit 1:1
Enter bit 2:0
Enter bit 3:0
Enter bit 4:1
Initial Codeword: [1, 0, 0, 1, 0, 0, 0]
Codeword to be transmitted is: [1, 0, 0, 1, 1, 1, 0]
Introduce Error - Y -> YES N->NO:N
The message received is: [1, 0, 0, 1, 1, 1, 0]
No Error
Successfully Transferred
```

Run 2: error introduced at position 1

Terminal

```
$ python3 exp6.py
Enter the number of bits:4
Enter bit 1:1
Enter bit 2:0
Enter bit 3:0
Enter bit 4:1
Initial Codeword: [1, 0, 0, 1, 0, 0, 0]
Codeword to be transmitted is: [1, 0, 0, 1, 1, 1, 0]
Introduce Error - Y -> YES N->NO:Y
Enter the position to change bit:1
The message received is: [0, 0, 0, 1, 1, 1, 0]
Error!
```

The initial codeword `[1, 0, 0, 1, 0, 0, 0]` is the data `1001` followed by three zeros; after the division the transmitted codeword is `[1, 0, 0, 1, 1, 1, 0]`, i.e. the data plus the CRC `110`, exactly as in the worked example.

The 16-bit CRC-CCITT generator

To obtain CRC-CCITT only the generator has to be replaced (the data must then contain at least 17 bits). The same program, run with 20 data bits `10110100111001011001`, gives:

exp6.py (first line replaced)

```
div = [1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1]      # x^16 + x^12 + x^5 + 1
```

CRC-CCITT, no error

```
$ python3 exp6.py
Enter the number of bits:20
...
Enter bit 20:1
...
Codeword to be transmitted is: [1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0,
1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0]
Introduce Error - Y -> YES N->NO:N
The message received is: [1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0,
0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0]
No Error
Successfully Transferred
```

CRC-CCITT, bit 5 flipped

```
$ python3 exp6.py
Enter the number of bits:20
...
Enter bit 20:1
...
Codeword to be transmitted is: [1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0,
1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0]
Introduce Error - Y -> YES N->NO:Y
Enter the position to change bit:5
The message received is: [1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0,
0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0]
Error!
```

Observations

- Without an error the remainder at the receiver is all zeros and the program prints *No Error / Successfully Transferred*.
- After a single bit of the codeword is flipped the remainder is non-zero and the program prints *Error!*; the same happens with the 16-bit generator.
- The CRC is appended as r extra bits (3 bits for `1011`, 16 bits for CRC-CCITT), whatever the length of the data.

> **Result.** A Python program was written that appends a CRC to the data using modulo-2 division, lets the user corrupt one bit, and checks the received codeword. The data `1001` gave the codeword `1001110`; the error-free codeword was accepted and the codeword with the first bit flipped was reported as erroneous.

Viva voce questions for this experiment: see Appendix A, page 49.

EXPERIMENT NO. 7 Date: 07/10/2026

Frame Sorting Technique Used in Buffers

Aim

> Write a program for the frame sorting technique used in buffers.

Theory

Fragmentation, sequence numbers and the receive buffer

A message that does not fit into one frame is cut into pieces, and every piece is sent in a frame of its own. So that the receiver can put the pieces together again, each frame carries a *sequence number*. On a real network the frames do not always arrive in the order in which they were sent: they may take different paths, wait in queues or be retransmitted after an error (as in Selective Repeat). The receiver therefore keeps the frames it has received in a *buffer*, sorts them by sequence number and hands the data to the upper layer only when it is in the correct order.

Frames and sorting in the program

- **Sender:** the message is cut into pieces of `DATA_SZ = 3` characters (the last piece may be shorter) and the pieces are numbered 1, 2, 3, . . . ; a frame is the pair (sequence number, data).
- **Channel:** arrival in the wrong order is imitated by swapping every frame with a randomly chosen frame.
- **Receiver:** the buffered frames are sorted with *bubble sort* and the data parts are joined to give the message.

Bubble sort compares neighbouring frames and exchanges them when the left one has the larger sequence number; after the i-th pass the i largest sequence numbers are in their final places. For n frames it makes $n(n-1)/2$ comparisons, and the number of exchanges equals the number of pairs of frames that are out of order. This is acceptable for the small buffers used here.

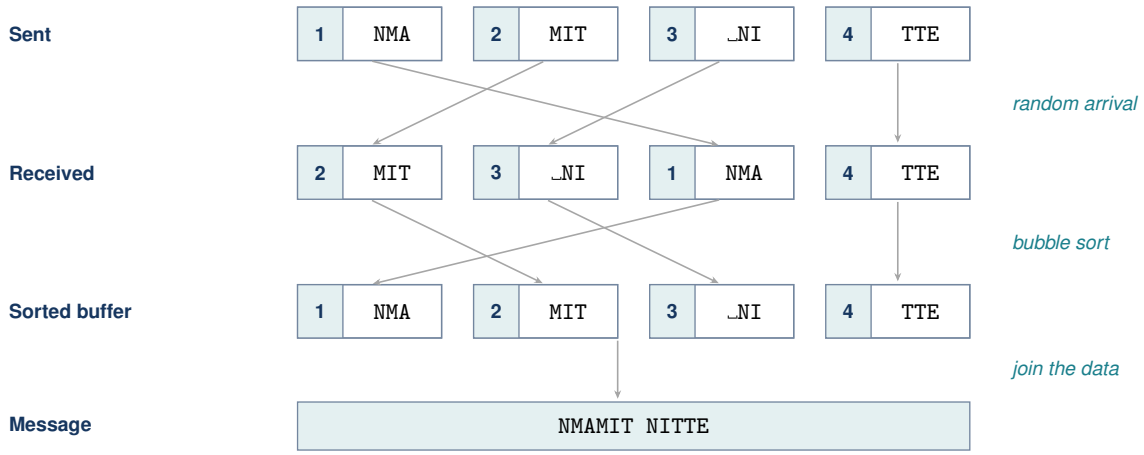


Figure 14. Frames of the message `NMAMIT NITTE`: sent, received in random order and sorted.

Algorithm

Fragmentation and shuffling

1. Read the message `msg`.
2. Cut `msg` into pieces of `DATA_SZ = 3` characters; the last piece may be shorter.
3. Number the pieces 1, 2, 3, ...; each frame is a pair (number, piece). Display the fragmented frames.
4. For every position `i`, choose a random position `j` and swap `frames[i]` with `frames[j]`.
5. Display the shuffled frames.

Frame sorting (receive buffer)

1. Let `n` be the number of frames in the buffer.
2. For `i = 0, ..., n-1` and for `j = 0, ..., n-i-2`: if `frames[j] > frames[j+1]`, swap the two frames.
3. Display the sorted frames.
4. Join the data parts of the frames in this order and display the sorted message.

For the received order 2 3 1 4 shown in Figure 14 the bubble sort works as follows:

Pass	Comparisons	Swaps	Order after the pass
1	3	1	2 1 3 4
2	2	1	1 2 3 4
3	1	0	1 2 3 4
4	0	0	1 2 3 4
Total	6	2	

Table 8: passes of the bubble sort for the received order 2 3 1 4

Program

exp7.py — frame sorting with bubble sort

```
import random
DATA_SZ = 3

def bubb_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

msg = input("Enter the message:")
msg_chunks = [msg[i:i + DATA_SZ] for i in range(0, len(msg), DATA_SZ)]
frames = list(enumerate(msg_chunks, start=1))
print("Fragmented frames:", frames)
for i in range(len(frames)):
    j = random.randint(0, len(frames) - 1)
    frames[i], frames[j] = frames[j], frames[i]
print("Shuffled frames:", frames)
bubb_sort(frames)
print("Sorted frames:", frames)
print("Sorted message:" + "".join(x[1] for x in frames))
```

Execution and output

Terminal

```
$ python3 exp7.py
Enter the message:NMAMIT NITTE
Fragmented frames: [(1, 'NMA'), (2, 'MIT'), (3, ' NI'), (4, 'TTE')]
Shuffled frames: [(2, 'MIT'), (3, ' NI'), (1, 'NMA'), (4, 'TTE')]
Sorted frames: [(1, 'NMA'), (2, 'MIT'), (3, ' NI'), (4, 'TTE')]
Sorted message:NMAMIT NITTE
```

The order of the shuffled frames is random and changes from run to run, but the sorted frames and the sorted message are always the same. Further test runs (one run for each message), including messages whose length is not a multiple of three and a message with ten frames:

Input message	Frames	Order after shuffling	Restored
NMAMIT NITTE	4	2 3 1 4	yes
HELLO	2	2 1	yes
HI	1	1	yes
COMPUTER NETWORKS	6	1 2 6 4 3 5	yes
COMPUTER NETWORKS LABORATORY	10	1 7 3 6 10 8 9 5 4 2	yes

Table 9: results of the program for five input messages (“yes”: the sorted message equals the input)

Observations

- The 12-character message is cut into four frames of three characters; the blank between MIT and NITTE belongs to the data of frame 3 (' NI') and is restored with it.
- Whatever the order of arrival, the bubble sort brings the frames back to 1, 2, 3, 4 and the joined data is the original message, because the sequence numbers are unique.
- For four frames the bubble sort makes $4 \cdot 3/2 = 6$ comparisons whatever the order, because the program always carries out every pass; the number of swaps (2 here) equals the number of pairs of frames that are out of order.
- If the length of the message is not a multiple of DATA_SZ the last frame is shorter (HELLO gives HEL and LO). With ten frames the sequence numbers are compared as integers, so frame 10 is placed after frame 9.

Result. A Python program was written that cuts a message into numbered frames of three characters, shuffles them to imitate arrival in the wrong order and restores the order with bubble sort on the sequence numbers. The message NMAMIT NITTE was cut into four frames, received in the order 2, 3, 1, 4 and sorted back to NMAMIT NITTE.

Viva voce questions for this experiment: see Appendix A, page 49.

EXPERIMENT NO. 8 Date: _____

Client–Server File Transfer Using TCP Sockets

Aim

> Using TCP/IP sockets, write a client–server program in which the client sends the name of a file and the server sends back the contents of the requested file if it is present.

Theory

Sockets and ports

A *socket* is one end point of a two-way communication link between two programs on a network: what one program writes to its socket can be read by the program at the other end. A TCP/IP socket is identified by an IP address, a transport protocol (TCP or UDP) and a port number (0–65535). The client must know the pair (IP address, port) of the server. A socket has to be *bound* to a port before it can receive data from a remote program; a client socket is given a temporary (ephemeral) port by the operating system when it connects.

Stream sockets (TCP)

A stream socket (`SOCK_STREAM`) uses TCP, which is connection-oriented and reliable: a connection is set up with a three-way handshake, lost segments are retransmitted and the data arrives in order and without duplicates. TCP delivers a *byte stream* and does not preserve message boundaries: one `send()` may be received by several `recv()` calls and the other way round. A call `recv(n)` returns *at most* n bytes, and it returns an empty bytes object when the other side has closed the connection. The receiver therefore reads in a loop until the connection is closed. Sockets carry bytes, not text: strings are converted with `encode()` before sending and with `decode()` after receiving.

Protocol of this experiment

The client sends the file name (one short message). The server answers with the bytes of the file and then closes the connection; if the file does not exist, the answer is the text `File not found`. The closing of the connection tells the client that the answer is complete.

Socket calls

Step	Server	Client
1	`socket()` creates a TCP socket	`socket()` creates a TCP socket
2	`bind()` attaches it to (IP address, port)	–
3	`listen()` makes it a listening socket	–
4	`accept()` waits for a client and returns a new socket	`connect()` starts the handshake
5	`recv()` receives the file name	`sendall()` sends the file name
6	`sendall()` sends the file contents	`recv()` in a loop receives them
7	`close()` closes the client's socket	`close()` closes the socket

Table 10: socket calls of the TCP server and the TCP client in the order in which they are made

Why `sendall()` and a `recv()` loop

`send()` may hand only a part of the data to the network and returns the number of bytes it accepted; `sendall()` repeats the call until all the bytes have been sent or an error occurs. On the receiving side the loop collects the pieces: a file of 2592 bytes is received by `recv(1024)` as pieces of 1024, 1024 and 544 bytes (Section "Observations").

Communication diagram

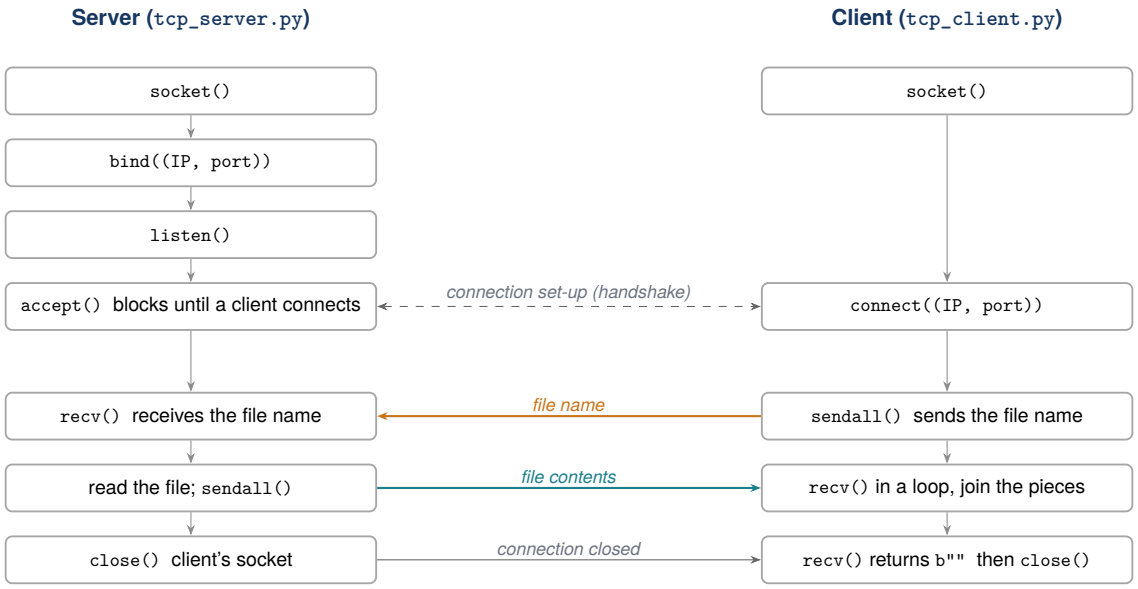


Figure 15. Sequence of socket calls in Experiment 8 (the server is started first and keeps running).

Algorithm

Server side

1. Start.
2. Create a TCP socket with `socket(AF_INET, SOCK_STREAM)` and allow the port to be reused.
3. `bind()` the socket to (127.0.0.1, 8888) and call `listen()`.
4. Wait for a client with `accept()`; it returns a new socket for that client.
5. Receive the file name with `recv()`.
6. If the file exists, read it and send all its bytes with `sendall()`; otherwise send `File not found`.
7. Close the client's socket and go back to step 4 (stop with Ctrl+C).

Client side

1. Start.
2. Create a TCP socket.
3. `connect()` to the server's address; if the server is not running, print a message and stop.
4. Read the file name and send it with `sendall()`.
5. Call `recv()` repeatedly and join the pieces until it returns an empty bytes object.
6. If the data is `File not found`, display that message; otherwise display the contents and save them as `received_`*name*.
7. Close the socket. Stop.

Program

tcp_server.py — TCP server

```python
# tcp_server.py - sends the requested file to the client
import socket
import os

HOST = "127.0.0.1"               # this computer only (localhost)
PORT = 8888                      # port number the server listens on
BUFFER_SIZE = 1024               # maximum bytes received by one recv()

# create a TCP socket, bind it to (HOST, PORT) and start listening
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((HOST, PORT))
server_socket.listen(1)
print("Server is listening on", HOST, "port", PORT)

try:
    while True:
        client_socket, client_address = server_socket.accept()
        with client_socket:
            print("Accepted connection from", client_address)

            # receive the file name sent by the client
            file_name = client_socket.recv(BUFFER_SIZE).decode()
            print("Received file name =", file_name)

            # send the contents if the file exists, otherwise an error text
            if os.path.isfile(file_name):
                with open(file_name, "rb") as file:
                    client_socket.sendall(file.read())
                print("File sent")
            else:
                client_socket.sendall(b"File not found")
                print("File not found")
except KeyboardInterrupt:
    print("\nServer stopped")
finally:
    server_socket.close()
```

tcp_client.py — TCP client

```python
# tcp_client.py - requests a file from the server and saves it
import socket
import sys
import os

SERVER_ADDRESS = ("127.0.0.1", 8888)    # (IP address, port) of the server
BUFFER_SIZE = 1024

# create a TCP socket and connect to the server
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    client_socket.connect(SERVER_ADDRESS)
except ConnectionRefusedError:
    sys.exit("Could not connect to the server. Is tcp_server.py running?")

file_name = input("Enter the file name: ").strip()
if file_name == "":
    client_socket.close()
    sys.exit("No file name entered.")

# send the file name, then read until the server closes the connection
client_socket.sendall(file_name.encode())
file_contents = b""
```

```
while True:
    chunk = client_socket.recv(BUFFER_SIZE)
    if not chunk:                       # empty bytes: the server has closed
        break
    file_contents += chunk
client_socket.close()

if file_contents == b"File not found":
    print("File not found on the server.")
else:
    print("File contents:")
    print(file_contents.decode(errors="replace"))
    save_name = "received_" + os.path.basename(file_name)
    with open(save_name, "wb") as file:
        file.write(file_contents)
    print("File received and saved as '" + save_name + "'")
```

Execution and output

Open two terminals in the folder that contains the programs. Create the test file `sample.txt` (three lines of text) and start the server first; it waits in `accept()` until a client connects.

Terminal 1 — server

```
$ python3 tcp_server.py
Server is listening on 127.0.0.1 port 8888
Accepted connection from ('127.0.0.1', 48040)
Received file name = sample.txt
File sent
Accepted connection from ('127.0.0.1', 48042)
Received file name = missing.txt
File not found
Accepted connection from ('127.0.0.1', 48058)
Received file name = big.txt
File sent
^C
Server stopped
```

Terminal 2 — client: a file that exists

```
$ cat sample.txt
Computer Networks Laboratory
Experiment 8: file transfer using TCP sockets
This file was sent by the server.
$ python3 tcp_client.py
Enter the file name: sample.txt
File contents:
Computer Networks Laboratory
Experiment 8: file transfer using TCP sockets
This file was sent by the server.

File received and saved as 'received_sample.txt'
$ cmp sample.txt received_sample.txt && echo "files are identical"
files are identical
```

Terminal 2 — client: a missing file and a larger file

```
$ python3 tcp_client.py
Enter the file name: missing.txt
File not found on the server.
$ python3 tcp_client.py
Enter the file name: big.txt
```

```
File contents:
line 1
line 2
line 3
   ... (lines 4 to 298 are not printed here) ...
line 299
line 300

File received and saved as 'received_big.txt'
$ cmp big.txt received_big.txt && echo "files are identical"; wc -c big.txt received_big.txt
files are identical
2592 big.txt
2592 received_big.txt
5184 total
```

If the client is started while the server is not running, `connect()` fails and the client reports it:

Terminal 2 — client: server not running

```
# the server has been stopped with Ctrl+C; now start only the client
$ python3 tcp_client.py
Could not connect to the server. Is tcp_server.py running?
```

Observations

- **File present.** The three lines of `sample.txt` (109 bytes) arrive in one `recv()`, are displayed and are saved as `received_sample.txt`; `cmp` shows that the copy is identical.
- **File absent.** The server replies `File not found`; the client prints *File not found on the server* and saves nothing.
- **Larger file.** `big.txt` has 2592 bytes. With `recv(1024)` it arrives in three pieces (1024, 1024 and 544 bytes) followed by an empty result when the server closes; the loop joins them and the saved file is byte for byte identical to the original.
- **Server not running.** The client reports *Could not connect to the server* instead of stopping with an error message of Python.
- The server handles one client at a time (an *iterative* server): the next client is served after the connection of the previous one has been closed.

> **Result.** A TCP client–server program was written in Python. The client sent a file name and the server returned the contents of the file; a missing file was reported with an error message. The files were received completely and unchanged (109 and 2592 bytes) with the sequence `socket()`, `bind()`, `listen()`, `accept()`, `recv()`, `sendall()`, `close()` at the server and `socket()`, `connect()`, `sendall()`, `recv()`, `close()` at the client.

Viva voce questions for this experiment: see Appendix A, page 50.

EXPERIMENT NO. 9 Date: _____

Client–Server Calculator Using UDP Sockets

Aim

Using UDP sockets, write a client–server program in which the client sends two numbers and an operator and the server responds with the result. Display the result and appropriate messages for invalid inputs at the client side.

Theory

Datagram sockets (UDP)

A datagram socket (`SOCK_DGRAM`) uses UDP. UDP is *connectionless*: there is no handshake, no `connect()`, `listen()` or `accept()`. Every message is an independent *datagram* that carries the address of its sender. UDP is *unreliable*: a datagram may be lost, duplicated or delivered out of order, and nothing is acknowledged or retransmitted. UDP is *message oriented*: one `sendto()` produces one datagram and one `recvfrom()` returns exactly one datagram, so message boundaries are preserved. The largest UDP payload over IPv4 is 65 507 bytes (65 535 minus the 20-byte IP header and the 8-byte UDP header).

Socket calls

The server creates a datagram socket and binds it to its port; `recvfrom()` blocks until a datagram arrives and returns the data together with the address of the sender, and the answer is sent back with `sendto()` to exactly that address. The client does not need to bind: the operating system chooses a source port at the first `sendto()`. Because a request or a reply can be lost, or the server may not be running, the client sets a time-out so that `recvfrom()` does not wait for ever.

Protocol and checking of the input

The request is one text datagram `number operator number` (for example `12 + 5`); the reply is the result as text or a text beginning with `Error:`. The client reads the three items with `input()` and checks them *before* anything is sent: both operands must be finite numbers and the operator one of `+ - * / %`. The server in addition protects itself against a division or modulo by zero and against a malformed request, so that wrong data never stops it.

Why `input()` and not command-line arguments? In a command such as `python3 udp_client.py 6 * 3` the shell replaces `*` by the list of file names of the folder before Python starts. Reading the operator with `input()` avoids this.

Communication diagram

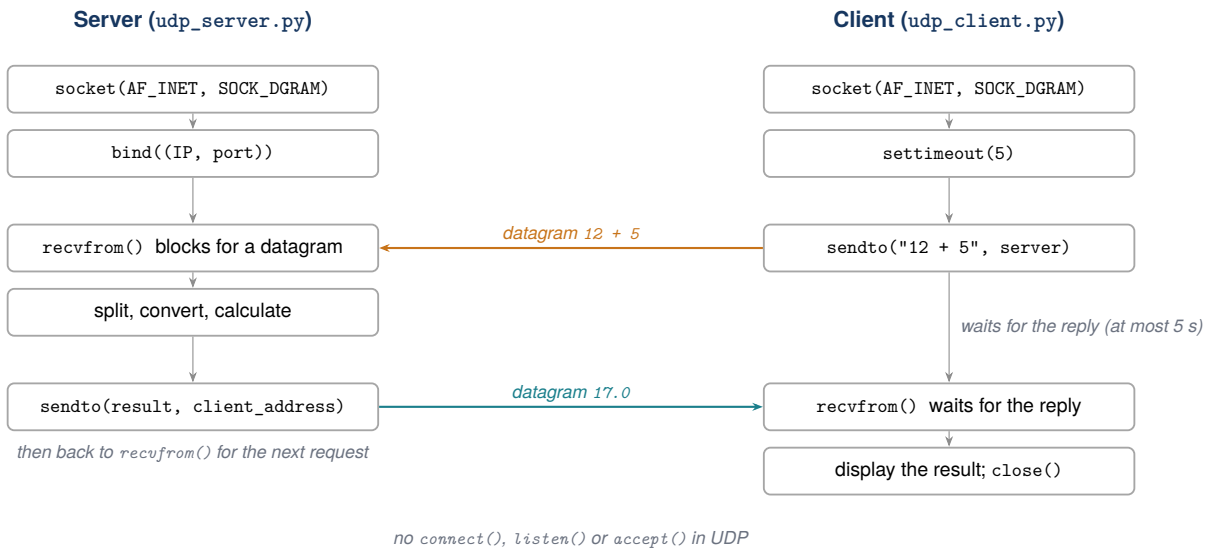


Figure 16. Sequence of socket calls in Experiment 9; every arrow between the columns is one datagram.

Algorithm

Server side

1. Start.
2. Create a UDP socket with `socket(AF_INET, SOCK_DGRAM)`.
3. `bind()` the socket to (127.0.0.1, 20001).
4. Wait with `recvfrom()`; it returns the request and the client's address.
5. Split the request into number, operator and number and convert the numbers with `float()`.
6. Calculate the result; for a division or modulo by zero, a wrong operator or a malformed request the result is an error text.
7. Send the result to the client's address with `sendto()` and go back to step 4.

Client side

1. Start.
2. Read the first number, the operator and the second number.
3. If an operand is not a number or the operator is not one of `+ - * / %`, display a message and stop.
4. Create a UDP socket and set a time-out of 5 seconds.
5. Send `number operator number` to the server's address with `sendto()`.
6. Receive the reply with `recvfrom()` and display it; if the time-out expires, display that there is no reply.
7. Close the socket. Stop.

Program

`udp_server.py` — **UDP server**

```
# udp_server.py - receives "number operator number" and replies with the result
import socket

SERVER_IP = "127.0.0.1"
SERVER_PORT = 20001
```

```
BUFFER_SIZE = 1024

# create a UDP socket and bind it to (SERVER_IP, SERVER_PORT)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind((SERVER_IP, SERVER_PORT))
print("UDP server up and listening on port", SERVER_PORT)

try:
    while True:
        # wait for a datagram; recvfrom() also tells us who sent it
        data, client_address = server_socket.recvfrom(BUFFER_SIZE)
        request = data.decode(errors="replace")
        print("Request from", client_address, ":", request)

        # work out the result (or an error message)
        try:
            num1, operator, num2 = request.split()
            a = float(num1)
            b = float(num2)
            if operator == "+":
                result = a + b
            elif operator == "-":
                result = a - b
            elif operator == "*":
                result = a * b
            elif operator == "/":
                if b == 0:
                    result = "Error: division by zero"
                else:
                    result = a / b
            elif operator == "%":
                if b == 0:
                    result = "Error: modulo by zero"
                else:
                    result = a % b
            else:
                result = "Error: invalid operator"
        except ValueError:
            result = "Error: invalid request"

        print("Result to client:", result)
        server_socket.sendto(str(result).encode(), client_address)
except KeyboardInterrupt:
    print("\nServer stopped")
finally:
    server_socket.close()
```

udp_client.py — UDP client

```
# udp_client.py - reads two numbers and an operator, asks the server to calculate
import socket
import sys
import math

SERVER_ADDRESS = ("127.0.0.1", 20001)   # (IP address, port) of the server
BUFFER_SIZE = 1024

def is_number(text):
    # True only for finite numbers such as 12, -3.5 or 1e3
    try:
        return math.isfinite(float(text))
    except ValueError:
        return False

# read and check the input before anything is sent
num1 = input("Enter first number : ").strip()
operator = input("Enter operator (+ - * / %) : ").strip()
num2 = input("Enter second number : ").strip()
```

```
20
21 if not is_number(num1) or not is_number(num2):
22     sys.exit("Invalid input: both operands must be numbers.")
23 if operator not in ("+", "-", "*", "/", "%"):
24     sys.exit("Invalid operator: use one of + - * / %")
25
26 # create a UDP socket; wait at most 5 seconds for the reply
27 client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
28 client_socket.settimeout(5)
29
30 request = num1 + " " + operator + " " + num2
31 client_socket.sendto(request.encode(), SERVER_ADDRESS)
32
33 try:
34     reply, server_address = client_socket.recvfrom(BUFFER_SIZE)
35     print(request, "=", reply.decode())
36 except (socket.timeout, ConnectionResetError):
37     print("No reply from the server. Is udp_server.py running?")
38 finally:
39     client_socket.close()
```

Execution and output

Start the server in the first terminal and run the client in the second terminal; the client asks for the first number, the operator and the second number.

Terminal 1 — server

```
$ python3 udp_server.py
UDP server up and listening on port 20001
Request from ('127.0.0.1', 42284) : 12 + 5
Result to client: 17.0
Request from ('127.0.0.1', 42871) : 20 / 4
Result to client: 5.0
Request from ('127.0.0.1', 55705) : 7 % 3
Result to client: 1.0
Request from ('127.0.0.1', 55629) : 2.5 * -4
Result to client: -10.0
Request from ('127.0.0.1', 59562) : 5 / 0
Result to client: Error: division by zero
^C
Server stopped
```

Terminal 2 — client: valid requests

```
$ python3 udp_client.py
Enter first number : 12
Enter operator (+ - * / %) : +
Enter second number : 5
12 + 5 = 17.0
$ python3 udp_client.py
Enter first number : 20
Enter operator (+ - * / %) : /
Enter second number : 4
20 / 4 = 5.0
$ python3 udp_client.py
Enter first number : 7
Enter operator (+ - * / %) : %
Enter second number : 3
7 % 3 = 1.0
$ python3 udp_client.py
Enter first number : 2.5
Enter operator (+ - * / %) : *
```

```
Enter second number : -4
2.5 * -4 = -10.0
$ python3 udp_client.py
Enter first number : 5
Enter operator (+ - * / %) : /
Enter second number : 0
5 / 0 = Error: division by zero
```

Invalid input is detected at the client; nothing is sent to the server (the server window above shows only the five requests that were valid):

Terminal 2 — client: invalid inputs (rejected at the client)

```
$ python3 udp_client.py
Enter first number : abc
Enter operator (+ - * / %) : +
Enter second number : 5
Invalid input: both operands must be numbers.
$ python3 udp_client.py
Enter first number : 8
Enter operator (+ - * / %) : ^
Enter second number : 2
Invalid operator: use one of + - * / %
```

If the server is not running, the client waits for the time-out and then reports it:

Terminal 2 — client: server not running

```
# the server has been stopped; the client waits 5 seconds and then reports it
$ python3 udp_client.py
Enter first number : 12
Enter operator (+ - * / %) : +
Enter second number : 5
No reply from the server. Is udp_server.py running?
```

Observations

No.	Number 1	Operator	Number 2 and result displayed at the client	Detected by
1	12	+	5 → `12 + 5 = 17.0`	server (calculation)
2	20	/	4 → `20 / 4 = 5.0`	server (calculation)
3	7	%	3 → `7 % 3 = 1.0`	server (calculation)
4	2.5	*	−4 → `2.5 * -4 = -10.0`	server (calculation)
5	1e3	−	0.5 → `1e3 - 0.5 = 999.5`	server (calculation)
6	5	/	0 → `Error: division by zero`	server
7	7	%	0 → `Error: modulo by zero`	server
8	abc	+	5 → `Invalid input: both operands must be numbers.`	client
9	8	^	2 → `Invalid operator: use one of + - * / %`	client

Table 11: test cases of Experiment 9 and the message that is displayed at the client

- The result is always shown as a decimal number (`17.0`) because the operands are converted with `float()`; decimal, negative and exponent forms (`2.5`, `-4`, `1e3`) are accepted.

- A division or modulo by zero is caught by the server, which sends the text `Error: …` instead of a number; the client simply displays what it receives.
- Wrong operands or a wrong operator are caught by the client before the datagram is sent, so the server never sees them; a datagram of any other form that reaches the server (for example the text `hello`) is answered with `Error: invalid request` and the server keeps running.
- The server answers one request after the other; the client port (for example 42284) is chosen by the operating system and is different for every run.
- Without a reply the client does not wait for ever: after 5 seconds it reports that the server did not answer.

Result. A UDP client–server program was written in Python. The client sent two numbers and an operator in one datagram, the server returned the result (`12 + 5 = 17.0`) and wrong input was reported at the client (non-numeric operand, invalid operator, division or modulo by zero, no reply from the server).

Viva voce questions for this experiment: see Appendix A, page 50.

EXPERIMENT NO. 10 Date: 06/11/2026

Distance Vector Routing Algorithm

Aim

> Write a program in C for the distance vector routing algorithm to find a suitable path for transmission, and display the routing (distance) table of every router.

Theory

Routing and the routing table

Routing is the job of the network layer: every router must decide on which outgoing link to forward a packet so that it reaches its destination along a good path. The decision is taken from a *routing table*, which gives for every destination the *distance* (cost) to reach it and the *next hop* (the neighbour to which the packet is handed). The cost of a link can be its delay, its length in hops or any other metric; the program reads it as an integer.

Distance vector routing

In distance vector routing each router knows only the cost of the links to its own neighbours. It keeps a vector of its best known distances to all routers and periodically sends this vector to its neighbours. On receiving the vectors, the router recomputes every entry with the **Bellman–Ford equation**

$$D_x(y) \;=\; \min_{v \,\in\, \text{neighbours of } x} \left\{\, c(x,v) + D_v(y) \,\right\}$$

where $c(x,v)$ is the cost of the link from x to neighbour v and $D_v(y)$ is the distance from v to y announced by v. The neighbour v that gives the minimum becomes the next hop for y. After at most $n-1$ rounds of exchange the tables of a network with n routers stop changing (they *converge*). The method is distributed and simple and was used by RIP; its weakness is the slow reaction to a link failure, the *count-to-infinity* problem, which is reduced by split horizon and poison reverse.

How the program models the network

The cost matrix `dmat[i][j]` holds the direct link cost from router i to router j; a large value such as 999 stands for "no direct link" and the diagonal is forced to 0. Each router i has a structure `rt[i]` with two arrays: `dist[j]`, the best distance to j found so far, and `from[j]`, the next hop used for j. Routers are numbered from 1 and printed as letters (1 = A, 2 = B, …) by printing the character `i+64`.

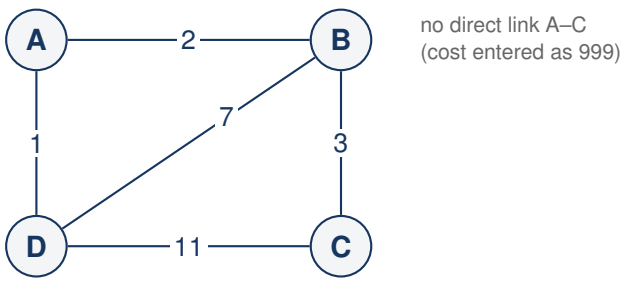


Figure 17. Four-router network used for the second test run (link costs on the edges).

Algorithm

1. Read the number of routers n and the $n \times n$ cost matrix `dmat`; set `dmat[i][i] = 0`.
2. Initialise every routing table with the direct costs, `rt[i].dist[j] = dmat[i][j]`, and the next hops, `rt[i].from[j] = j`.
3. Repeat $n - 1$ times (counter `c` from 1 to $n - 1$): for every router i, every destination j and every intermediate router k, if `dmat[i][k] + rt[k].dist[j]` is smaller than `rt[i].dist[j]`, replace the distance by the cost through k and set `rt[i].from[j] = k`.
4. For every router print its table: destination, next hop (*via*) and distance.

Program

exp10.c — distance vector routing

```c
#include <stdio.h>
#include <stdlib.h>
struct node
{
    int dist[20];
    int from[20];
} rt[10];
int main()
{
    int dmat[20][20];
    int n,i,j,k,c=1;
    printf("Enter the number of nodes: \n");
    scanf("%d", &n);
    printf("Enter the cost matrix: ");
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=n; j++)
        {
            scanf("%d", &dmat[i][j]);
            dmat[i][i]=0;
            rt[i].dist[j]=dmat[i][j];
            rt[i].from[j]=j;
        }
    }
    do
    {
        for(i=1; i<=n; i++)
            for(j=1; j<=n; j++)
                for(k=1; k<=n; k++)
                    if(rt[i].dist[j]>dmat[i][k] + rt[k].dist[j])
                    {
                        rt[i].dist[j]=rt[i].dist[k]+rt[k].dist[j];
                        rt[i].from[j]=k;
                    }
        c++;
    }
    while(c<n);
    for(i=1; i<=n; i++)
    {
        printf("\nDistance Table for router %c is in \n",i+64);
        for(j=1;j<=n;j++)
            printf("\tNode %d Via %d.  Distance: %d.\n",j,rt[i].from[j], rt[i].dist[j]);
    }
    return 0;
}
```

Execution and output

The program was compiled with `gcc` and run in the laboratory with three routers. Each row of the cost matrix is typed on one line; the diagonal value typed is replaced by 0 by the program.

Terminal — run 1 (three routers, as recorded in the laboratory)

```
$ gcc exp10.c -o exp10
$ ./exp10
Enter the number of nodes:
3
Enter the cost matrix: 1 0 1
2 5 3
3 1 0

Distance Table for router A is in
        Node 1 Via 1.  Distance: 0.
        Node 2 Via 2.  Distance: 0.
        Node 3 Via 3.  Distance: 1.

Distance Table for router B is in
        Node 1 Via 1.  Distance: 2.
        Node 2 Via 2.  Distance: 0.
        Node 3 Via 3.  Distance: 3.

Distance Table for router C is in
        Node 1 Via 1.  Distance: 3.
        Node 2 Via 2.  Distance: 1.
        Node 3 Via 3.  Distance: 0.
```

In this network every direct link is already the shortest path (for example A to C costs 1 directly, but $0 + 3 = 3$ through B), so every router reaches each destination directly.

> **Why a second run?** To see the algorithm choose paths through other routers, the program was run again on the network of Figure 17. The value 999 marks the missing link A–C.

Terminal — run 2 (four routers of Figure 17)

```
$ ./exp10
Enter the number of nodes:
4
Enter the cost matrix: 0 2 999 1
2 0 3 7
999 3 0 11
1 7 11 0

Distance Table for router A is in
        Node 1 Via 1.  Distance: 0.
        Node 2 Via 2.  Distance: 2.
        Node 3 Via 2.  Distance: 5.
        Node 4 Via 4.  Distance: 1.

Distance Table for router B is in
        Node 1 Via 1.  Distance: 2.
        Node 2 Via 2.  Distance: 0.
        Node 3 Via 3.  Distance: 3.
        Node 4 Via 1.  Distance: 3.

Distance Table for router C is in
        Node 1 Via 2.  Distance: 5.
        Node 2 Via 2.  Distance: 3.
        Node 3 Via 3.  Distance: 0.
        Node 4 Via 2.  Distance: 6.
```

```
Distance Table for router D is in
        Node 1 Via 1.  Distance: 1.
        Node 2 Via 1.  Distance: 3.
        Node 3 Via 1.  Distance: 6.
        Node 4 Via 4.  Distance: 0.
```

Observations

Router	Destination	Direct cost	Final route (next hop, distance)
A	C	999 (no link)	A → B → C (via B, 5)
B	D	7	B → A → D (via A, 3)
C	D	11	C → B → A → D (via B, 6)
D	C	11	D → A → B → C (via A, 6)

Table 12: entries of run 2 that differ from the direct link costs

- Every router starts with only its direct link costs and ends with the least cost to every other router; the next hop (*via*) is the first router on that path.
- A path through several routers can be cheaper than a direct link: B reaches D at cost 3 through A instead of 7 directly, and C reaches D at cost 6 instead of 11.
- The missing link A–C (999) is replaced by the path through B with cost $2 + 3 = 5$.
- The tables are symmetric when the cost matrix is symmetric (A to C and C to A both cost 5).
- The update loop is repeated $n - 1$ times, which is enough for the longest shortest path of a network with n routers.

Result. A C program for the distance vector routing algorithm was written and executed. For each router the program computed the least-cost distance and the next hop to every destination; in the four-router network the routes A → C (via B, cost 5) and D → C (via A, cost 6) were found although no direct path of that cost exists.

Viva voce questions

Q. What is distance vector routing? Each router keeps a vector of distances to all destinations and shares it with its neighbours; routes are recomputed with the Bellman–Ford equation.

Q. How does it differ from link state routing? In link state routing every router learns the whole topology and runs Dijkstra's algorithm; in distance vector routing a router only knows what its neighbours tell it.

Q. What is the count-to-infinity problem? After a link fails, routers keep raising their distances using each other's old values, so bad news spreads slowly; split horizon and poison reverse reduce it.

Q. Why is the loop repeated $n - 1$ times? A shortest path in a network of n routers has at most $n - 1$ links, so $n - 1$ rounds are enough for all distances to settle.

Q. Why is 999 entered for a missing link? It acts as infinity: it is larger than any real path cost, so it is replaced as soon as a path through another router is found.

Q. What does `i+64` print? With `%c` the ASCII code 65 is printed as A, 66 as B, and so on, so router 1 is shown as A.

APPENDIX A

Viva Voce Question Bank

Short questions with model answers for each experiment, for revision before the viva and the laboratory examination.

Experiment 1 — Point-to-Point Network: Queue Size, Bandwidth and Packet Drops

Q. What is NS-2 and which languages does it use?
Ans. NS-2 is a discrete-event network simulator. The simulation engine is written in C++ and the scenario is described in OTcl (Tcl) scripts.

Q. What do the event symbols `+`, `-`, `r` and `d` in the trace file mean?
Ans. Enqueue, dequeue, receive and drop respectively.

Q. Why is no packet dropped at 1 Mb but many at 0.25 Mb?
Ans. The CBR source offers $500 \times 8/0.005 = 800$ kb/s. A 1 Mb link can carry this; a 0.25 Mb link cannot, so its queue fills and DropTail discards the new arrivals.

Q. What is a DropTail queue and what is the default queue limit in NS-2?
Ans. A FIFO queue that drops the arriving packet when it is full; the default limit is 50 packets.

Q. Why does a larger queue reduce the number of drops?
Ans. The extra buffer space holds part of the excess traffic, so fewer packets arrive at a full queue (at the cost of a larger queuing delay).

Experiment 2 — TCP and UDP Traffic: Packet Counts and Throughput

Q. State the main differences between TCP and UDP.
Ans. TCP is connection-oriented and reliable with flow and congestion control; UDP is connectionless, unreliable and has no congestion control, so its rate is set only by the application.

Q. What are the roles of `Agent/TCPSink` and `Agent/Null`?
Ans. `TCPSink` receives TCP data and returns the acknowledgements; `Null` just absorbs UDP packets without any reply.

Q. What does `Application/FTP` do in NS-2?
Ans. It simulates a bulk file transfer, i.e. an unlimited supply of data for the TCP agent.

Q. Why does the UDP count hardly change with the bandwidth while the TCP count does?
Ans. The CBR source has a fixed rate (800 kb/s); TCP adapts its window to the capacity that remains.

Q. How does `exp2_throughput.awk` compute the throughput?
Ans. It sums the sizes of all received packets and divides by the time between the first enqueue and the last receive, multiplied by $8/1000$ to obtain kb/s.

Q. Which link is the bottleneck here?
Ans. The link n2–n3, because the traffic of both flows has to pass through it.

Experiment 3 — Ethernet LAN: Throughput versus Data Rate and Error Rate

Q. What is CSMA/CD and how does a station react to a collision?
Ans. Carrier Sense Multiple Access with Collision Detection: a station listens before and during transmission; on a collision it stops, and retries after a random delay chosen by binary exponential backoff.

Q. What does the `make-lan` command create?
Ans. A shared LAN (one common channel) of the listed nodes, giving each node an LL, an interface queue and the specified MAC, here `Mac/802_3`.

Q. What is the purpose of `$tcp set class_ 1` and `$ns color 1 red`?
Ans. `class_` sets the flow id of the packets; `color` makes NAM draw the packets of flow 1 in red.

Q. Why does the throughput fall when the error rate is increased?
Ans. Lost packets must be retransmitted and TCP shrinks its congestion window after each loss, so less data is delivered in the same time.

Q. Why does the throughput hardly increase when the LAN rate is raised from 2 Mb to 3 Mb?
Ans. The LAN is no longer the bottleneck; the TCP window and the round-trip time limit the sending rate.

Experiment 4 — Congestion Window of TCP Vegas and Reno

Q. What is the congestion window?
Ans. The sender-side limit on the amount of unacknowledged data; it grows when acknowledgements arrive and is reduced when congestion is detected.

Q. Distinguish slow start from congestion avoidance.
Ans. In slow start the window doubles every round-trip time; in congestion avoidance it increases by about one packet per round-trip time.

Q. How do TCP Reno and TCP Vegas detect congestion?
Ans. Reno reacts to packet loss; Vegas watches the round-trip delay and adjusts the window before losses occur.

Q. What do `$tcp1 attach $tfile1` and `$tcp1 trace cwnd_` do?
Ans. They connect the agent to an open file and log the variable `cwnd_` to it every time it changes.

Q. What is a collision in Ethernet and how is it handled?
Ans. Two stations transmit at the same time and the frames overlap; CSMA/CD detects it, both stations stop, and each retransmits after a random backoff delay.

Q. Why is the Vegas curve flat after about 3 s?
Ans. Vegas has reached the window at which the queue it creates in the network stays between its thresholds, so it no longer changes the window.

Experiment 5 — Bit Stuffing in the Data-Link Layer

Q. Why is stuffing needed in the data-link layer?
Ans. To prevent the flag pattern that delimits a frame from appearing inside the data, so that the receiver finds the frame boundaries unambiguously.

Q. What is the HDLC flag and what is the bit-stuffing rule?
Ans. The flag is `01111110`; the sender inserts a 0 after every five consecutive 1s of data.

Q. How does the receiver recognise a stuffed bit?
Ans. A 0 that directly follows five consecutive 1s is a stuffed bit and is removed.

Q. What is the worst-case overhead of bit stuffing?
Ans. When the data is all 1s: one extra bit per five data bits, i.e. 20 %.

Q. How does byte stuffing differ?
Ans. It works on whole bytes: a flag or escape byte in the data is preceded by an escape byte (as in PPP).

Experiment 6 — Error Detection Using CRC (CRC-CCITT)

Q. What is a CRC and how is the check value calculated?
Ans. A polynomial code: the message with r appended zeros is divided by the generator polynomial using modulo-2 arithmetic and the r-bit remainder is the CRC.

Q. Write the generator polynomial of CRC-CCITT.
Ans. $x^{16} + x^{12} + x^5 + 1$ (`0x11021`, 17 bits).

Q. Why does a zero remainder at the receiver indicate no error?
Ans. The transmitted codeword is an exact multiple of $G(x)$; an error pattern $E(x)$ that is not a multiple of $G(x)$ leaves a non-zero remainder.

Q. Which errors can a CRC not detect?
Ans. Error patterns that are themselves divisible by the generator polynomial.

Q. How many zeros are appended before the division?
Ans. Degree of the generator, i.e. `len(div) - 1`: 3 here, 16 for CRC-CCITT.

Q. What is modulo-2 arithmetic?
Ans. Arithmetic without carries in which both addition and subtraction are the XOR operation.

Experiment 7 — Frame Sorting Technique Used in Buffers

Q. Why must the frames in the receive buffer be sorted?
Ans. Frames can arrive out of order (different paths, delays, retransmissions), but the upper layer needs the data in the original order; so the receiver buffers the frames and sorts them by sequence number.

Q. What is the purpose of the sequence number?
Ans. It gives the position of the frame in the message, so the receiver can restore the order, detect missing frames and discard duplicates.

Q. How does bubble sort work and how many comparisons does it make for n frames?
Ans. It compares neighbouring frames and swaps them when they are out of order, so the largest remaining sequence number moves to the end in every pass. It makes $n(n-1)/2$ comparisons.

Q. What does `enumerate(msg_chunks, start=1)` do?
Ans. It pairs every piece of the message with a counter that starts at 1, giving frames such as `(1, 'NMA')`.

Q. What happens if the length of the message is not a multiple of `DATA_SZ`?
Ans. The last piece, and so the last frame, is shorter; it is numbered and sorted like the others.

Experiment 8 — TCP Sockets: Client–Server File Transfer

Q. What is a socket and how is a TCP socket identified?
Ans. A socket is one end point of a two-way communication between two programs. A TCP/IP socket is identified by an IP address, the protocol (TCP) and a port number; a connection by the pair of such end points.

Q. Name the calls of the TCP server and of the TCP client in order.
Ans. Server: `socket()`, `bind()`, `listen()`, `accept()`, `recv()`, `sendall()`, `close()`. Client: `socket()`, `connect()`, `sendall()`, `recv()`, `close()`.

Q. What do `bind()`, `listen()` and `accept()` do?
Ans. `bind()` attaches the socket to an IP address and a port; `listen()` makes it a passive socket that queues incoming connections (the argument is the backlog); `accept()` blocks until a client connects and returns a new socket for that client together with its address.

Q. Why does `accept()` return a new socket?
Ans. The listening socket stays open to accept further clients; the new socket is the private channel to one client.

Q. Why must the client call `recv()` in a loop?
Ans. TCP is a byte stream without message boundaries and `recv(n)` returns at most n bytes. An empty bytes object means that the server has closed the connection, i.e. the file is complete.

Q. What is the difference between `send()` and `sendall()`?
Ans. `send()` may transmit only part of the data and returns the number of bytes sent; `sendall()` keeps sending until all the data has been sent or an error occurs.

Q. What is `SO_REUSEADDR` used for?
Ans. It lets the server bind to its port again at once after a restart. Without it `bind()` can fail with *Address already in use* while an old connection is still in the TIME_WAIT state.

Q. Why do we use `encode()`, `decode()` and the modes `"rb"` and `"wb"`?
Ans. Sockets carry bytes: text is encoded before sending and decoded after receiving; the binary file modes keep the contents of the file exactly as they are.

Q. Is this server concurrent?
Ans. No, it is iterative: it serves one client at a time. To serve several clients at once it would use threads or `select()`.

Experiment 9 — UDP Sockets: Client–Server Calculator

Q. How do UDP sockets differ from TCP sockets in the program?
Ans. `SOCK_DGRAM` instead of `SOCK_STREAM`; no `listen()`, `accept()` or `connect()`; the server only binds, and both sides use `sendto()` and `recvfrom()`.

Q. Why does a UDP server not call `listen()` and `accept()`?
Ans. There is no connection to accept: every datagram is independent and carries the address of its sender.

Q. How does the server know where to send the reply?
Ans. `recvfrom()` returns the data and the client's (IP address, port); the reply is sent to that address with `sendto()`.

Q. Why does the client set a time-out?
Ans. A datagram or its reply may be lost, or the server may not be running; without a time-out `recvfrom()` would wait for ever.

Q. Does UDP keep message boundaries? What is the largest payload?
Ans. Yes: one `sendto()` gives one datagram and one `recvfrom()` returns one datagram. The largest UDP payload over IPv4 is 65 507 bytes.

Q. Where are invalid inputs detected?
Ans. Non-numeric operands and a wrong operator are detected at the client before anything is sent; division or modulo by zero and malformed requests are caught by the server, which replies with an `Error:` text.

Q. Why is the result shown as `17.0` and not `17`?
Ans. `float()` converts the operands, so the arithmetic result is a float.

Q. Why is the operator read with `input()` and not as a command-line argument?
Ans. The shell would expand `*` as a wildcard before the program starts.

APPENDIX B

Troubleshooting Guide

The messages below were produced by the real tools (NS-2 2.35, `gawk`, `mawk`, Python 3) for the mistakes that are easiest to make when typing the programs. If a program does not run in the laboratory, compare it line by line with this list.

Message or symptom	Cause	Fix
NS-2 / Tcl		
`can't read "tracefile": no such variable`	A variable used inside `proc finish` is not listed in `global`.	`global ns namfile tracefile` (Experiment 3).
`can't read "cwnd1": no such variable`	`$cwnd1.tr` is read as the variable `cwnd1` followed by `.tr`. The channels were opened as `tfile1`, `tfile2`.	`global ... tfile1 tfile2` and `close $tfile1`, `close $tfile2` (Experiment 4).
`invalid command name "MAC/802_3"`	Class names are case sensitive.	Write `Mac/802_3`.
`invalid command name "8"`	A stray character on a line of its own: Tcl treats the first word of every line as a command.	Delete the stray character.
No message, but the LAN behaves differently and the throughput changes	The `make-lan` command was broken over two lines, so the last arguments were not part of the command.	Keep the whole `make-lan` command on *one* line.
No message, but packets are 210 bytes instead of 500	`packetSize_` was written as two words (`packet Size_`); NS-2 does not complain.	Write `$cbr0 set packetSize_ 500`.
A `%` prompt appears and nothing runs	`ns` was started without a script.	Run `ns exp1.tcl`; leave the prompt with Ctrl-D.
AWK		
`BEGIN blocks must have an action part` (gawk), `syntax error at or near end of line` (mawk)	In AWK the `{` of a `BEGIN` block must be on the same line as `BEGIN`.	Write `BEGIN {`.
`syntax error` (gawk), `missing ) near ;` (mawk)	Unbalanced parentheses, for example in the `Tput` formula.	Make every `(` match a `)`.
`invalid char ''' in expression` (gawk), `unexpected character '''` (mawk)	AWK strings are written with double quotes only.	`printf("%f\t %.2f\n", time, Tput);`
The XGraph window or the data file is empty	The `printf` line of the AWK script is commented out with `#`, so nothing is printed.	Remove the `#` in front of `printf`.
Throughput is negative (about −6200 kb/s in Experiment 3)	The test is `time < stopTime` instead of `time > stopTime`, so `stopTime` never increases.	`if (time > stopTime).`
Python		
`SyntaxError: expected '('` (older versions: `invalid syntax`)	A hyphen or a space in a name (`bit-stuffing`, `return destuffed data`).	Use underscores: `bit_stuffing`, `destuffed_data`.
`NameError: name 'FALSE' is not defined`	The Python constants are `True` and `False`.	`flag = False`.
`SyntaxError` at `if bit == `1':`	A backtick was typed instead of a quote.	`if bit == '1':`
Sockets (Experiments 8 and 9; Python 3.12 on Linux)		
`ConnectionRefusedError: [Errno 111] Connection refused` (Windows: `WinError 10061`)	The client was started before the server, the server has stopped, or the port numbers of the two programs differ.	Start the server first; use the same address and port in both programs. The client of Experiment 8 catches this error and prints a message.
`OSError: [Errno 98] Address already in use` (Windows: `WinError 10048`)	Another copy of the server is still running, or the port is held in TIME_WAIT after the server closed a connection first.	Stop the old server; keep `SO_REUSEADDR` before `bind()`; or use another port.

Message or symptom	Cause	Fix
`TypeError: a bytes-like object is required, not 'str'`	A string was given to `send()`, `sendall()` or `sendto()`.	Encode it first: `file_name.encode()`.
`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xff …`	Binary data was decoded as text.	Keep the data as bytes and write it with `"wb"`, or decode with `errors="replace"`.
The client prints *File contents: File not found* and saves a file	The reply was decoded to `str` but compared with a `bytes` value (`b'File not found'`), so the two are never equal.	Compare bytes with bytes, as in `tcp_client.py`.
Only the first 1024 bytes of a longer file arrive	One `recv(1024)` returns at most 1024 bytes; TCP has no message boundaries.	Call `recv()` in a loop until it returns `b""`.
The UDP client waits for ever	No time-out was set and the server is not running, has crashed, or the datagram was lost.	`settimeout(5)` and catch `socket.timeout`; check that `udp_server.py` is running.
`ValueError: too many values to unpack (expected 2)` or `not enough values to unpack` in a UDP server that splits the expression with a regular expression	Input such as `-5+3` or `abc` does not give exactly two numbers.	Read the numbers and the operator separately, split the request into three parts and catch `ValueError`.
`ZeroDivisionError: float modulo`	`%` with a divisor of 0 was not checked.	Test `b == 0` before `a % b`, as in `udp_server.py`.
`SyntaxWarning: invalid escape sequence '\+'` (Python 3.12)	A regular expression was written in an ordinary string.	Use a raw string: `r"[+\-*/%]"`.

APPENDIX C

Files and Quick Command Reference

Files used in each experiment

Exp.	Programs and data typed by the student	Files produced
1	`exp1.tcl`, `exp1.awk`, `plot_graph_1`, `plot_graph_2`	`out.tr`, `out.nam`
2	`exp2.tcl`, `exp2.awk`, `exp2_throughput.awk`, `plot_tcp`, `plot_udp`	`out.tr`, `out.nam`, `plot_throughput`
3	`exp3.tcl`, `exp3.awk`	`exp3.tr`, `exp3.nam`, `exp3.graph`
4	`exp4.tcl`, `exp4.awk`	`exp4.tr`, `exp4.nam`, `cwnd1.tr`, `cwnd2.tr`, `tcp1`, `tcp2`
5	`exp5.py`	—
6	`exp6.py`	—
7	`exp7.py`	—
8	`tcp_server.py`, `tcp_client.py`, `sample.txt` (any text file)	`received_sample.txt` (the copy made by the client)
9	`udp_server.py`, `udp_client.py`	—
10	`exp10.c`	`exp10` (compiled program)

Commands

Open a terminal in the folder that contains the programs and type:

Terminal

```
# Experiment 1
ns exp1.tcl
awk -f exp1.awk out.tr
xgraph plot_graph_1
xgraph plot_graph_2

# Experiment 2
ns exp2.tcl
awk -f exp2.awk out.tr
xgraph plot_tcp
xgraph plot_udp
awk -f exp2_throughput.awk out.tr > plot_throughput
xgraph plot_throughput

# Experiment 3  (exp3.graph is written by the finish procedure)
ns exp3.tcl
xgraph exp3.graph

# Experiment 4  (tcp1 and tcp2 are written by the finish procedure)
ns exp4.tcl
xgraph tcp1
xgraph tcp2

# Experiments 5, 6 and 7
python3 exp5.py
python3 exp6.py
python3 exp7.py
```

```
# Experiment 8  (two terminals in the same folder; start the server first)
python3 tcp_server.py        # terminal 1: waits for a client, stop with Ctrl+C
python3 tcp_client.py        # terminal 2: enter a file name, e.g. sample.txt

# Experiment 9  (two terminals; start the server first)
python3 udp_server.py        # terminal 1
python3 udp_client.py        # terminal 2: first number, operator, second number
```

Comparing several runs. Save the data file of every run under a different name (for example `cp exp3.graph rate_1Mb`) and give all of them to one XGraph command, `xgraph exp3.graph rate_1Mb`, to draw the curves in the same window.

Python on other systems. On Windows use `python` or `py` instead of `python3`. If the two socket programs run on two different computers, the server must listen on its own address (or `0.0.0.0`) instead of `127.0.0.1`, and the client must use the address of the server.